INF1005
Topics for Week 03:

- Intro to JavaScript
- Document Object Model (DOM)
- JavaScript Libraries & Frameworks
- TypeScript

JAVASCRIPT

- History, Language & Syntax
- Including JavaScript in HTML
- The Document Object Model (DOM)
- Event Handling
- Debugging JavaScript
- Advantages & Disadvantages of Client-side Scripting

JAVASCRIPT LIBRARIES & FRAMEWORKS

- Bootstrap JS
- jQuery
- AJAX
  - Now out of fashion, Fetch API (2015) is gaining popularity
- JSON
- TypeScript

Additional Materials for Self-study

Fundamentals of Web Development

Third Edition by Randy Connolly and Ricardo Hoar

Chapter 8

JavaScript 1:

Language Fundamentals

In this chapter you will learn . . .

- About JavaScript's role in contemporary web development
- How to add JavaScript code to your web pages
- The main programming constructs of the language
- The importance of objects and arrays in JavaScript
- How to use functions in JavaScript

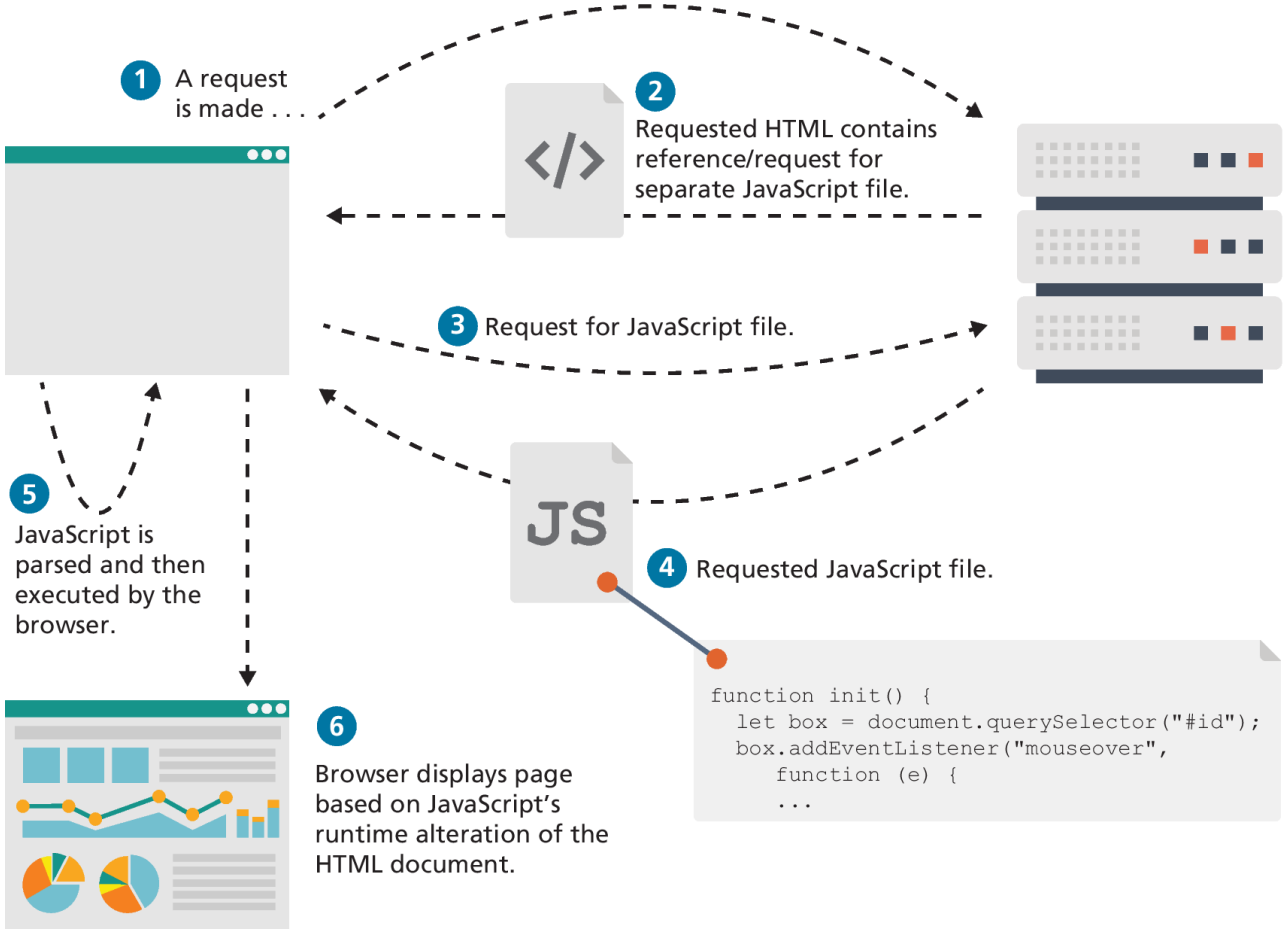
What Is JavaScript and What Can It Do?

- JavaScript: it is an object-oriented, dynamically typed scripting language
- *primarily* a client-side scripting language as well.
- variables are objects in that they have properties and methods
- Unlike more familiar object-oriented languages Such as Java, C#, and C++, functions in JavaScript are also objects.
- JavaScript is dynamically typed (also called weakly typed) in that variables can be easily (or implicitly) converted from one data type to another.

Client-Side Scripting

Client-side scripting refers to the client machine (i.e., the browser) running code locally rather than relying on the server to execute code and return the result.

A client machine downloads and executes JavaScript code



Client-Side Scripting: Advantages

- Processing can be off-loaded from the server to client machines, thereby reducing the load on the server.
- The browser can respond more rapidly to user events than a request to a remote server ever could, which improves the user experience.
- JavaScript can interact with the downloaded HTML in a way that the server cannot, creating a user experience more like desktop software than simple HTML ever could.

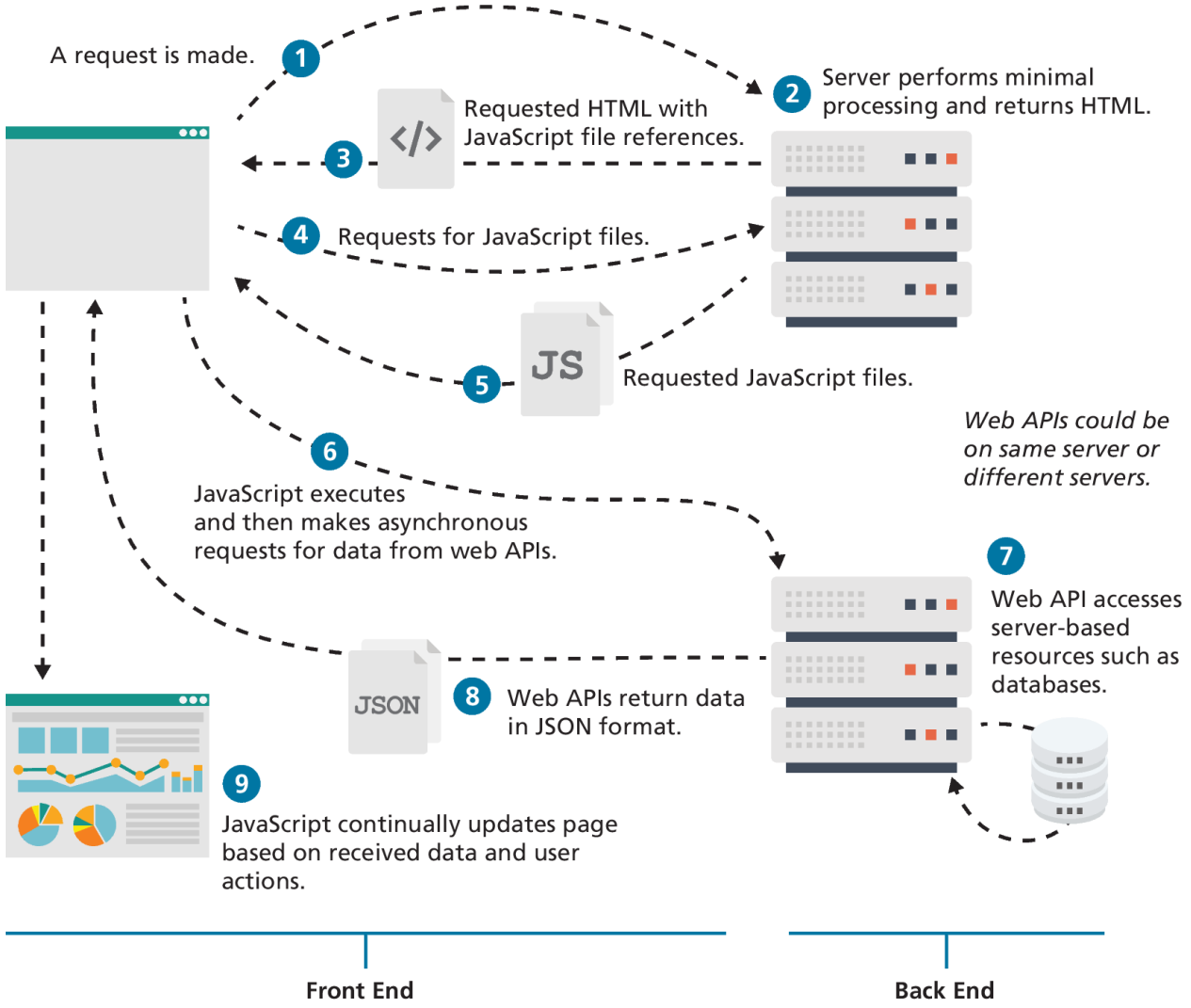
Client-Side Scripting: Disadvantages

- There is no guarantee that the client has JavaScript enabled, meaning any required functionality must be implemented redundantly on the server.
- JavaScript-heavy web applications can be complicated to debug and maintain.
- JavaScript is not fault tolerant. Browsers are able to handle invalid HTML or CSS. But if your page has invalid JavaScript, it will simply stop execution at the invalid line.
- While JavaScript is universally supported in all contemporary browsers, the language (and its APIs) is continually being expanded. As such, newer features of the language may not be supported in all browsers.

JavaScript's History

- JavaScript was introduced by Netscape in their Navigator browser back in 1996.
- Netscape submitted JavaScript to Ecma International in 1997, **ECMAScript** is simultaneously a superset and subset of the JavaScript programming language.
- The Sixth Edition (or **ES6**) was the one that introduced many notable new additions to the language (such as classes, iterators, arrow functions, and promises)
- The latest version of ECMAScript is the Tenth Edition (generally referred to as ES11 or ES2020)

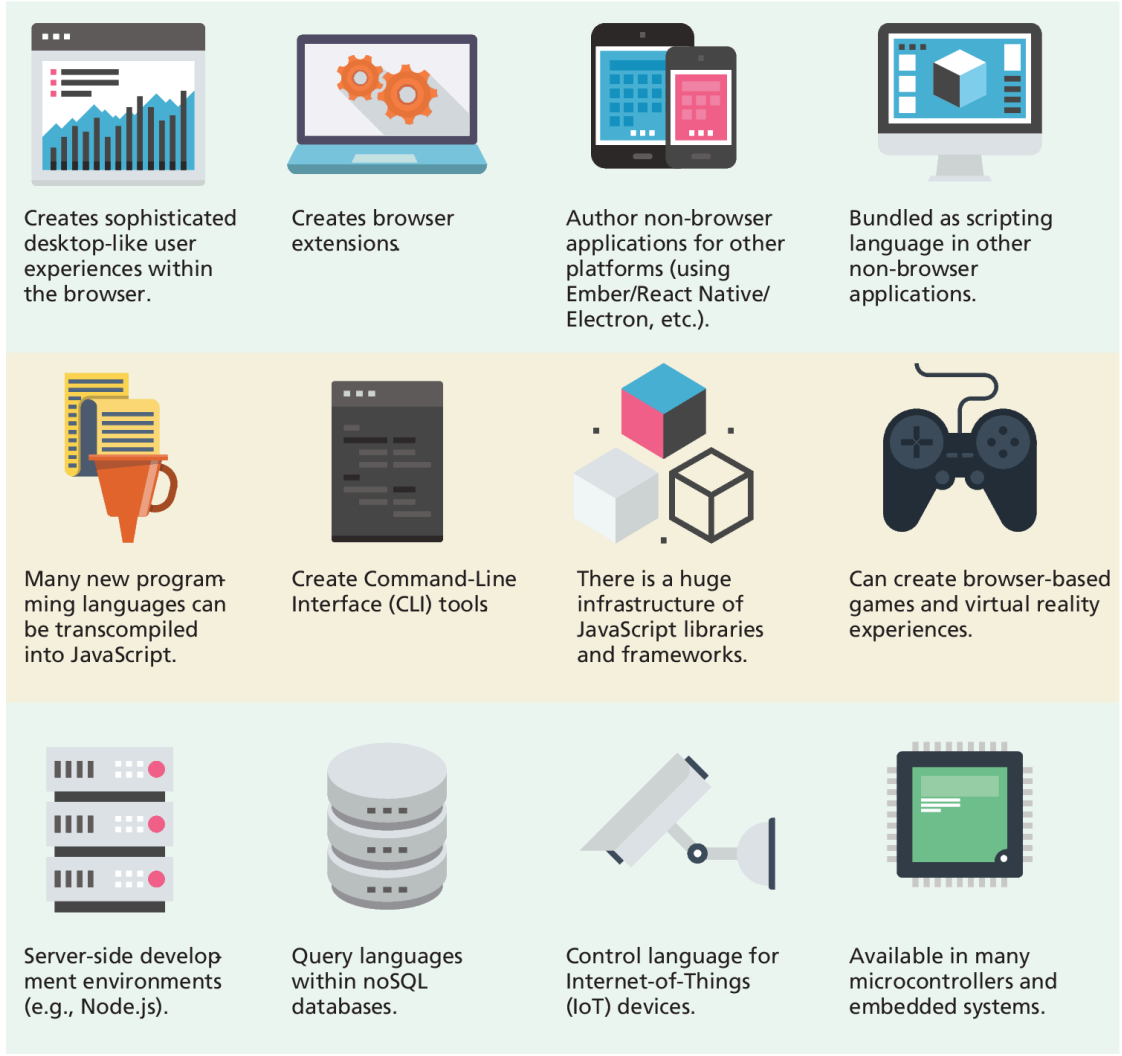
JavaScript and Web 2.0



JavaScript in Contemporary Software Development

JavaScript's role has expanded beyond the constraints of the browser

- It can be used as the language within server-side runtime environments such as Node.js.
- MongoDB use JavaScript as their query language
- Adobe Creative Suite and OpenOffice use JavaScript as their end-user scripting language
- ….



Where Does JavaScript Go?

Just as CSS styles can be inline, embedded, or external, JavaScript can be included in a number of ways.

- **Inline JavaScript** refers to the practice of including JavaScript code directly within some HTML element attributes.
- Embedded JavaScript refers to the practice of placing JavaScript code within a <script> element
- The recommended way to use JavaScript is to place it in an external file. You do this via the <script> tag

Adding JavaScript to a page

```
<html lang="en">
<head>
  <title>JavaScript placement possibilities</title>
  <script>
    /* A JavaScript Comment */
    alert("This will appear before any content");
  </script>

  <script src="greeting.js"></script>
</head>
<body>
<h1>Page Title</h1>

<a href="JavaScript:OpenWindow();">for more info</a>
<input type="button" onClick="alert('Are you sure?');" />

<script>
   alert("Hello World");
</script>
```

Embedded JavaScript

External JavaScript

Inline JavaScript

Embedded JavaScript

Users without JavaScript

Users have a myriad of reasons for not using JavaScript

- Search engines
- Browser extensions
- Text browser
- Accessible browser

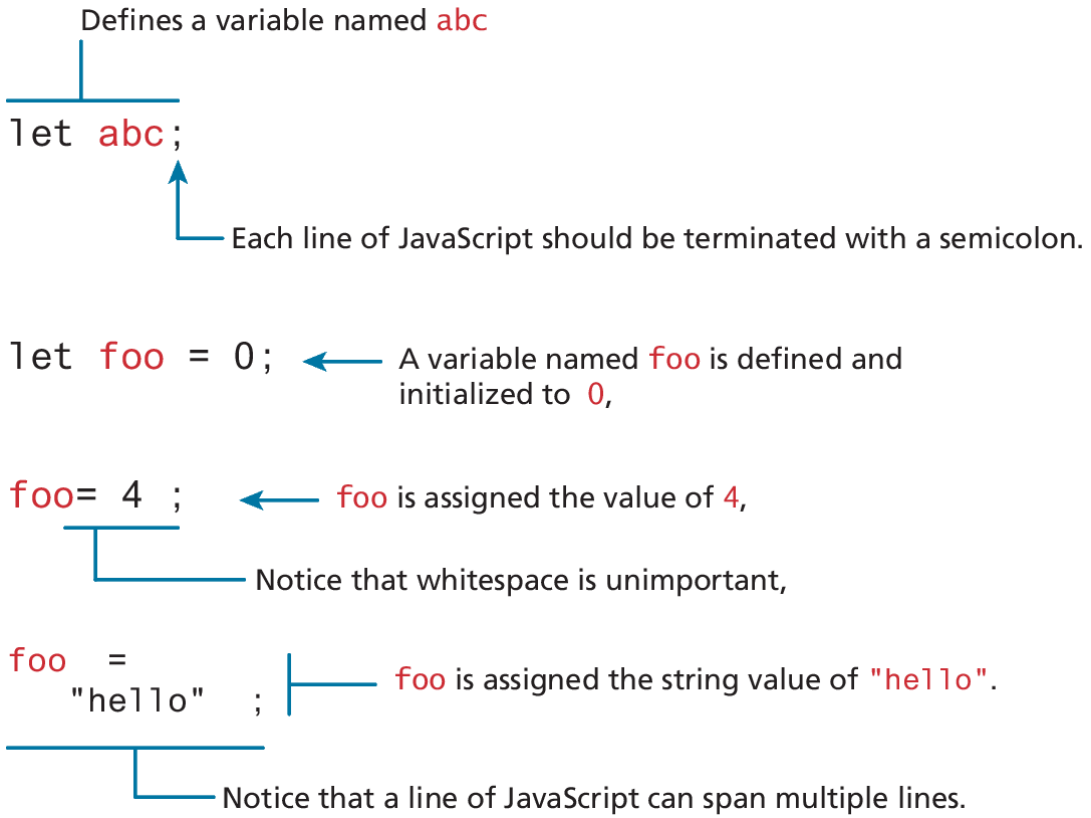
HTML provides an easy way to handle users who do not have JavaScript enabled: the <noscript> element. Any text between the opening and closing tags will only be displayed to users without the ability to load JavaScript.

Variables and Data Types

Variables in JavaScript are **dynamically typed**, meaning that you do not have to declare the type of a variable before you use it.

To declare a variable in JavaScript, use either the **var**, **const**, or **let** keywords (see Table 8.1)

Assignment can happen at declaration time by appending the value to the declaration, or at runtime with a simple right-to-left assignment

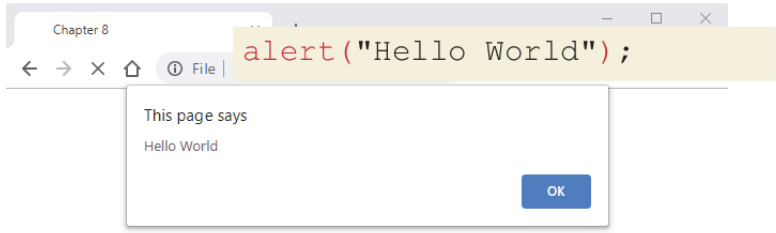


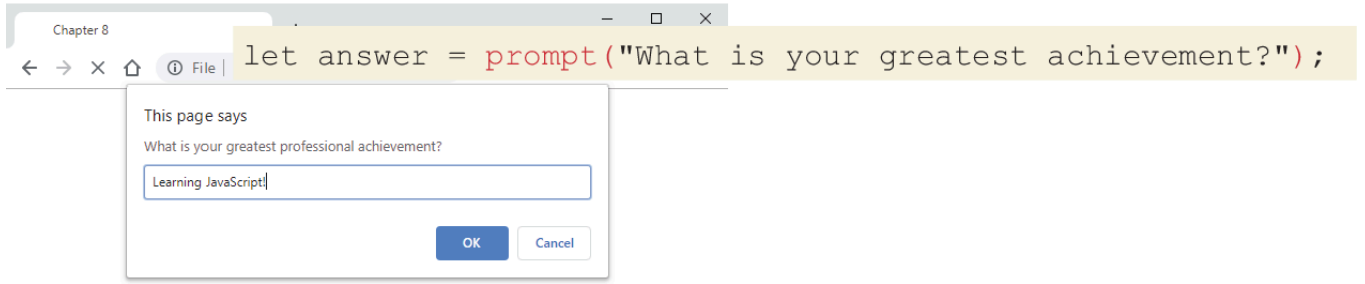
JavaScript Output

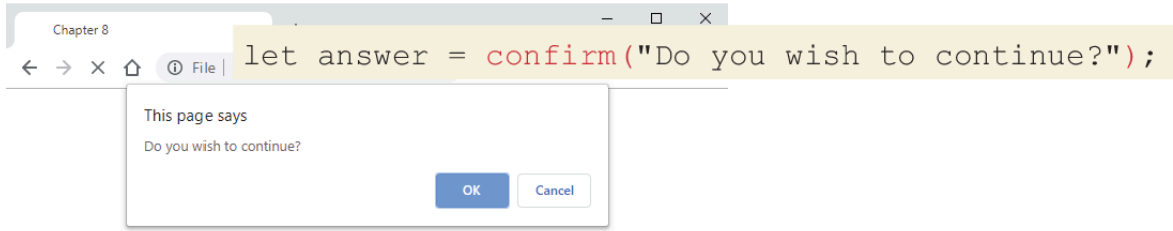
alert() Displays content within a browser-controlled pop-up/modal window.

prompt() Displays a message and an input field within a modal window.

confirm() Displays a question in a modal window with ok and cancel buttons.



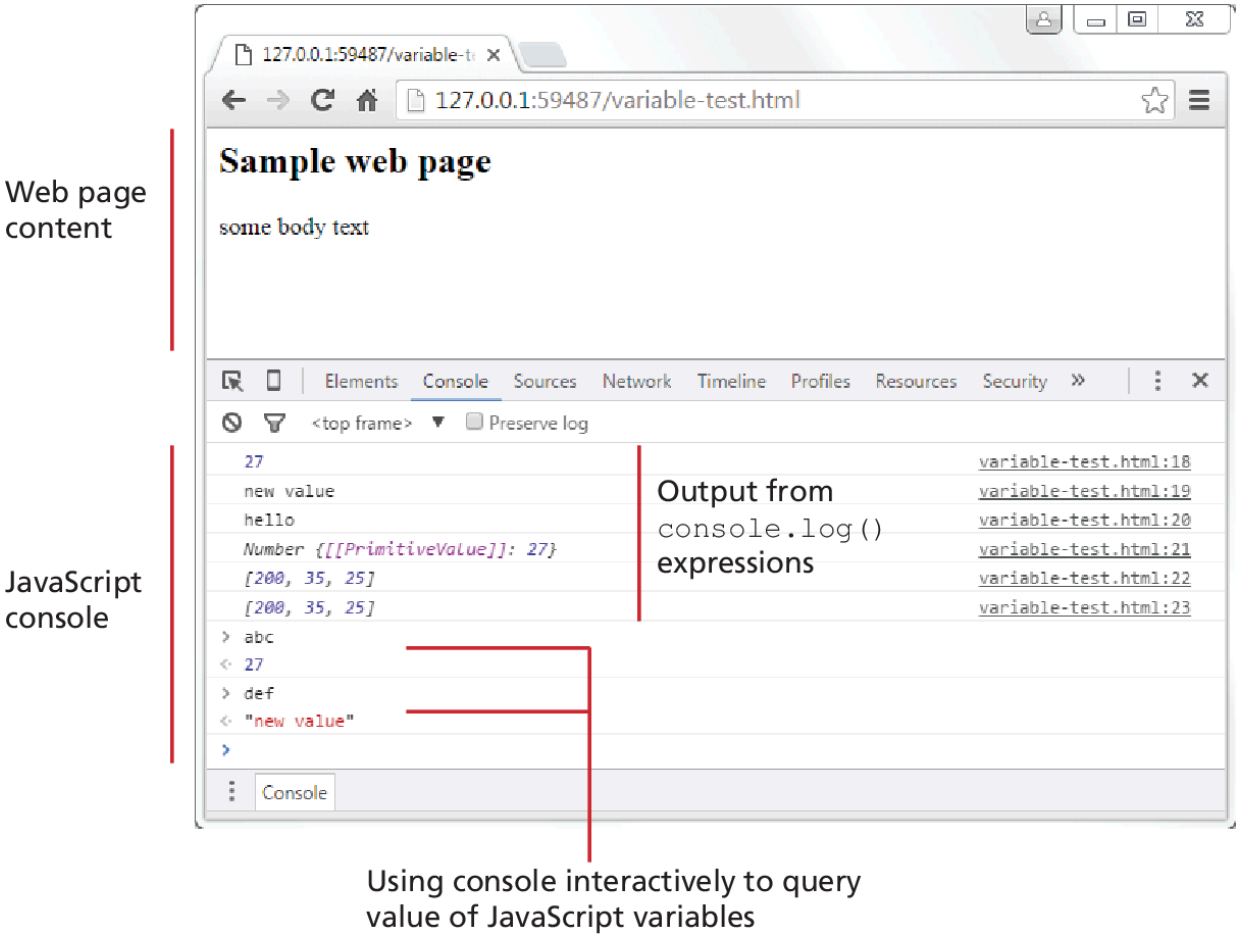




JavaScript Output (ii)

- **document.write()** Outputs the content (as markup) directly to the HTML document.

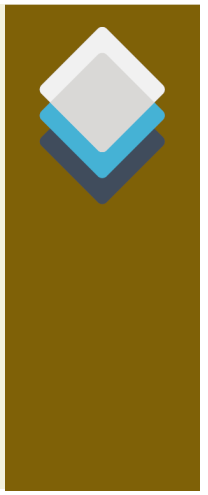
- **console.log()** Displays content in the browser's JavaScript console.



Document.write() note

NOTE

While several of the examples in this chapter make use of **document.write()**, the usual (and more trustworthy) way to generate content that you want to see in the browser window will be to use the appropriate JavaScript DOM (Document Model Object) method. You will learn how to do that in the next chapter.



Data Types

JavaScript has two basic data types:

- **reference types** (usually referred to as objects)
- **primitive types** (i.e., nonobject, simple types).
  - What makes things a bit confusing for new JavaScript developers is that the language lets you use primitive types as if they are objects.

Primitive Types

- **Boolean** True or false value.
- **Number** Represents some type of number. Its internal format is a double precision 64-bit floating point value.
- **String** Represents a sequence of characters delimited by either the single or double quote characters.
- **Null** Has only one value: **null**.
- **Undefined** Has only one value: **undefined**. This value is assigned to variables that are not initialized. Notice that undefined is different from null.
- **Symbol** New to ES2015, a symbol represents a unique value that can be used as a key value.

Primitive vs Reference Types

Primitive variables contain the value of the primitive directly within memory.

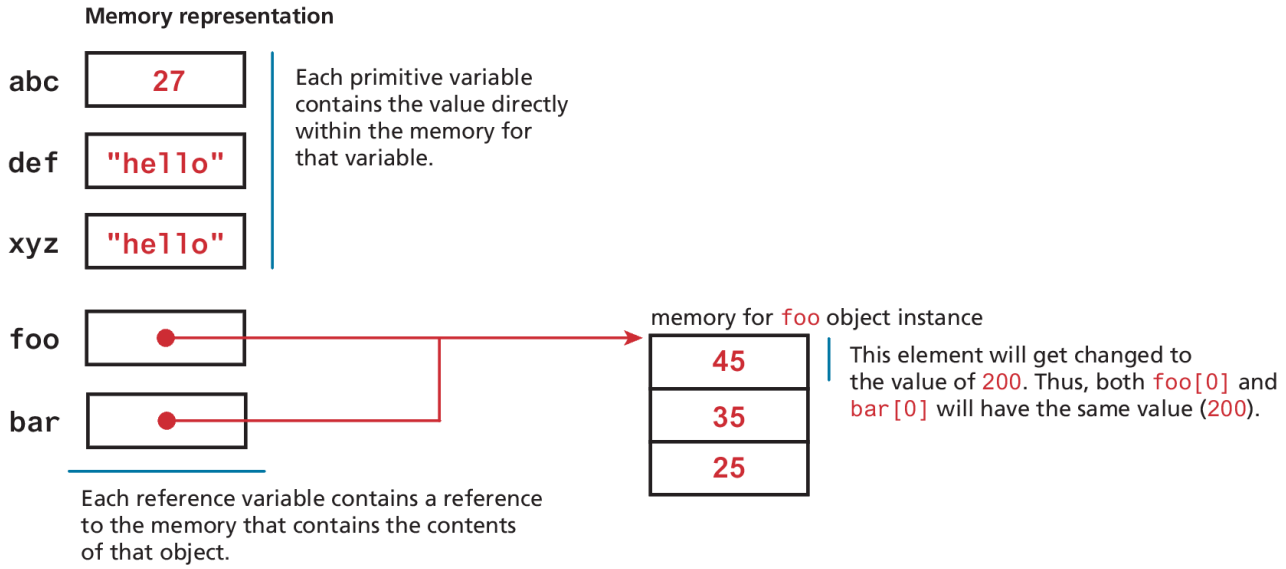
In contrast, object variables contain a reference or pointer to the block of memory associated with the content of the object.

```
let abc = 27;
let def = "hello";
```
variables with primitive types

```
let foo = [45, 35, 25];
```
variable with reference type (i.e., array object)

```
let xyz = def;
let bar = foo;
```
these new variables differ in important ways (see below)

```
bar[0] = 200;
```
changes value of the first element of array



Let vs const

All of these let examples work with no errors.

```
let abc = 27;
abc = 35;

let message = "hello";
message = "bye";

let msg = "hello";
msg = "hello";

let foo = [45, 35, 25];
foo[0] = 123;
foo[0] = "this is ok";

let person = {name: "Randy"};
person.name = "Ricardo";

person = {};
```

Some of these const examples work won't work, but some will work.

```
const abc = 27;
abc = 35;
```
Will generate runtime exception, since you cannot reassign a value defined with const.

```
const message = "hello";
message = "bye";
```
Will generate runtime exception.

```
const msg = "hello";
msg = "hello";
```
Will generate runtime exception.

```
const foo = [45, 35, 25];
foo[0] = 123;
foo[0] = "this is also ok";
```
You are allowed to change elements of an array, even if defined with a const keyword.

```
const person = {name: "Randy"};
person.name = "Ricardo";
```
Allowed to change properties of an object.

```
person = {};
```
Will generate runtime exception.

Built-In Objects

JavaScript has a variety of objects you can use at any time, such as arrays, functions, and the **built-in objects**.

Some of the most commonly used built-in objects include **Object**, **Function**, **Boolean**, **Error**, **Number**, **Math**, **Date**, **String**, and **Regexp**.

Later we will also frequently make use of several vital objects that are not part of the language but are part of the browser environment. These include the **document**, **console**, and **window** objects.

```
let def = new Date();
// sets the value of abc to a string containing the current date
let abc = def.toString();
```

Concatenation

To combine string literals together with other variables. Use the concatenate operator (+).

Alternative technique for concatenation is **template literals** (listing 8.2)

```
const country = "France";
const city = "Paris";
const population = 67;
const count = 2;

let msg = city + " is the capital of " + country;
msg += " Population of " + country + " is " + population;

let msg2 = population + count;

// what is displayed in the console?

console.log(msg);
//Paris is the capital of France Population of France is 67

console.log(msg2);
// 69
```

LISTING 8.1 Using the concatenate operator

Conditionals

JavaScript's syntax for conditional statements is almost identical to that of PHP, Java, or C++.

In this syntax the condition to test is contained within () brackets with the body contained in {} blocks. Optional **else if** statements can follow, with an **else** ending the branch.

JavaScript has all of the expected comparator operators (<, >, ==, <=, >=, !=, !==, ===) which are described in Table 8.4.

Switch statement

The **switch** statement is similar to a series of **if...else** statements.

There is another way to make use of conditionals: the **conditional operator** (also called the **ternary operator**).

```
switch (artType) {
   case "PT":
      output = "Painting";
      break;
   case "SC":
      output = "Sculpture";
      break;
   default:
      output = "Other";
}
// equivalent
if (artType == "PT") {
   output = "Painting";
} else if (artType == "SC") {
   output = "Sculpture";
} else {
   output = "Other";
}
```

LISTING 8.4 Conditional statement using switch and an equivalent if-else

The conditional (ternary) operator

```
/* conditional (ternary) assignment */
foo = (y==4) ? "y is 4" : "y is not 4";
```

Condition — Value if true — Value if false

```
/* equivalent to */
if (y==4) {
  foo = "y is 4";
}
else {
  foo = "y is not 4";
}
```

```
let tip  = isLargeGroup ? 0.25 : 0.15;
```

```
/* equivalent to */
let tip;
if (isLargeGroup) {
   tip = 0.25;
}
else {
   tip = 0.15;
}
```

```
let price  = isChild ? 5 : isSenior ? 7 : 9;
```

```
/* equivalent to */
let price;
if (isChild)
   price = 5;
else if (isSenior)
   price = 7;
else
   price = 9;
```

Truthy and Falsy

Everything in JavaScript has an inherent Boolean value.

In JavaScript, a value is said to be **truthy** if it translates to true, while a value is said to be **falsy** if it translates to false.

All values in JavaScript are truthy except false, null, "", '', 0, NaN, and undefined

While and do . . . while Loops

While and **do…while** loops execute nested statements repeatedly as long as the while expression evaluates to true.

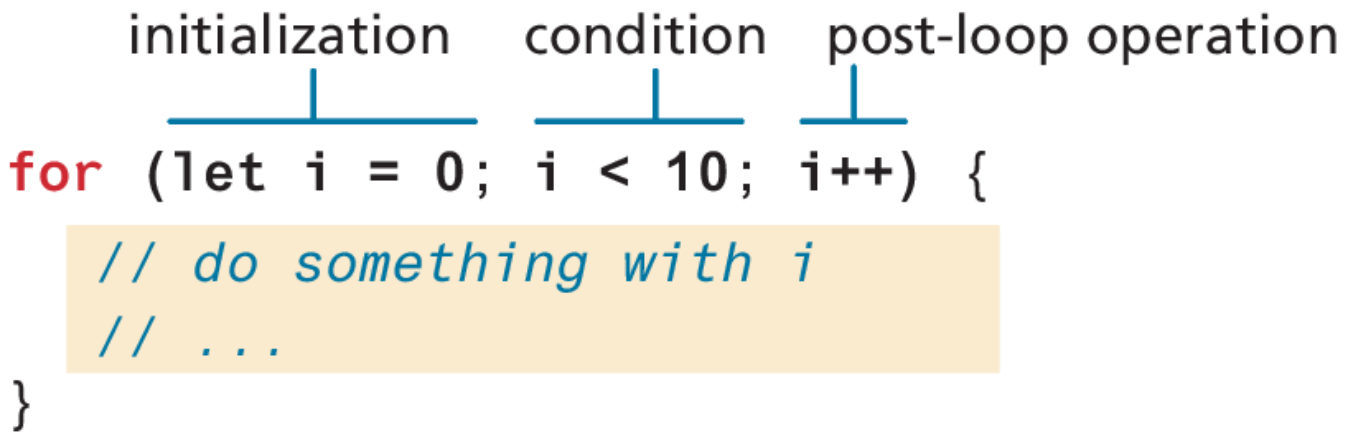
As you can see from this example, while loops normally initialize a **loop control variable** before the loop, use it in the condition, and modify it within the loop.

```
let count = 0;
while (count < 10) {
   // do something
   // ...
   count++;
}

count = 0;
do {
   // do something
   // ...
   count++;
} while (count < 10);
```

LISTING 8.5 While Loops

For Loops

For loops combine the common components of a loop—initialization, condition, and postloop operation—into one statement. This statement begins with the **for** keyword and has the components placed within () brackets, and separated by semicolons (;)

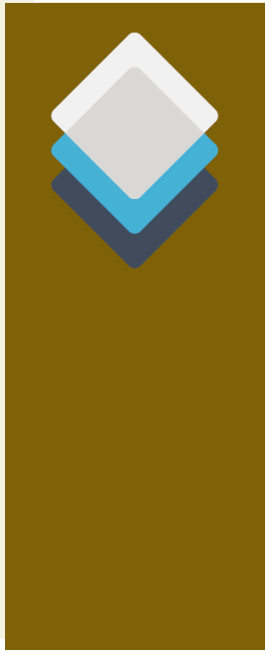


Infinite Loop Note

NOTE

Infinite loops can happen if you are not careful, and since the scripts are executing on the client computer, it can appear to them that the browser is “locked” while endlessly caught in a loop, processing.

Some browsers will even try to terminate scripts that execute for too long a time to mitigate this unpleasantness.



Try…catch

DIVE DEEPER

When the browser's JavaScript engine encounters a runtime error, it will throw an **exception**. These exceptions interrupt the regular, sequential execution of the program and can stop the JavaScript engine altogether. However, you can optionally catch these errors (and thus prevent the disruption) using the **try . . . catch block** as shown below.

```
try {
nonexistantfunction("hello");
}
catch(err) {
alert ("An exception was caught:" + err);
}
```

try...catch can also be used to your own error messages.

Arrays

Arrays are one of the most commonly used data structures in programming.

JavaScript provides two main ways to define an array.

First approach is **Array literal notation**, which has the following syntax:

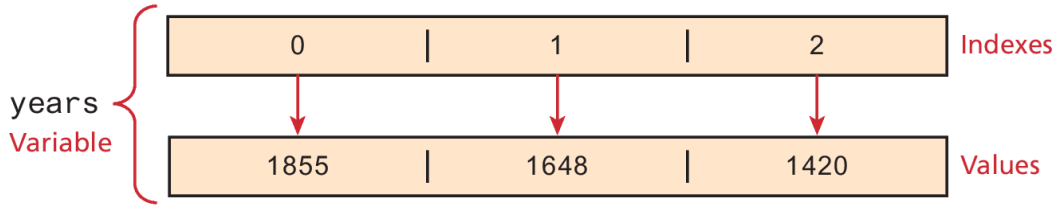
- const name = [*value1*, *value2*, ...];

The second approach is to use the Array() constructor:

- const name = new Array(*value1*, *value2*, ...);

Array example

```
const years = [1855, 1648, 1420];


const countries = ["Canada", "France",
"Germany", "Nigeria",
"Thailand", "United States"];

// arrays can also be multi-dimensional ... notice the commas!
const twoWeeks = [
["Mon","Tue","Wed","Thu","Fri"],
["Mon","Tue","Wed","Thu","Fri"]
];
// JavaScript arrays can contain different data types
const mess = [53, "Canada", true, 1420];
```



LISTING 8.6 Creating arrays using array literal notation

Iterating an array using for . . . of

ES6 introduced an alternate way to iterate through an array, known as the for...of loop, which looks as follows.

```
// iterating through an array
for (let yr of years) {
  console.log(yr);
}
```

```
//functionally equivalent to
for (let i = 0; i < years.length; i++) {
  let yr = years[i];
  console.log(yr);
}
```

Array Destructuring

Let's say you have the following array:

```
const league = ["Liverpool", "Man City", "Arsenal", "Chelsea"];
```

Now imagine that we want to extract the first three elements into their own variables. The "old-fashioned" way to do this would look like the following:

```
let first = league[0];
let second = league[1];
let third = league[2];
```

By using array destructuring, we can create the equivalent code in just a single line:

```
let [first,second,third] = league;
```

Objects

We have already encountered a few of the built-in objects in JavaScript, namely, arrays along with the Math, Date, and document objects.

In this section, we will learn how to create our own objects and examine some of the unique features of objects within JavaScript.

In JavaScript, **objects** are a collection of named values (which are called **properties** in JavaScript).

Unlike languages such as C++ or Java, objects in JavaScript are *not* created from classes. JavaScript is a prototype based language, in that new objects are created from already existing prototype objects, an idea that we will examine in Chapter 10.

Object Creation Using Object Literal Notation

The most common way is to use **object literal notation** (which we also saw earlier with arrays)

An object is represented by a list of key-value pairs with colons between the key and value, with commas separating key-value pairs.

To reference this object's properties, we can use either dot notation or square bracket notation.

```
const objName = {
      name1: value1,
      name2: value2,
      // ...
      nameN: valueN
};
```

```
objName.name1
objName["name1"]
```

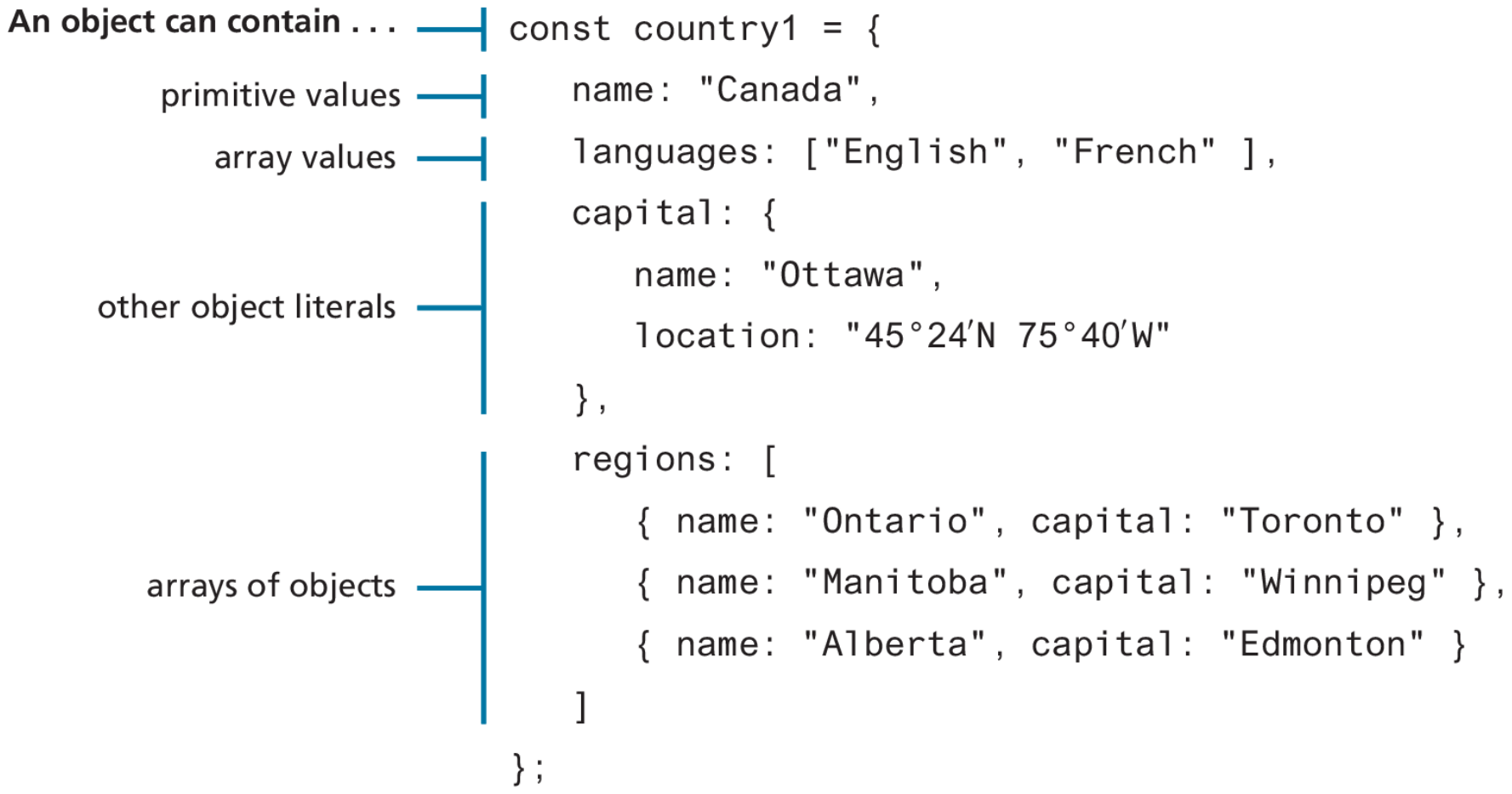
Object Creation Using Object Constructor

Another way to create an instance of an object is to use the Object constructor, as shown in the following:

```
// first create an empty object
const objName = new Object();

// then define properties for this object
objName.name1 = value1;
objName.name2 = value2;
```

Generally speaking, object literal notation is preferred in JavaScript over the constructed form.

Objects containing other content



Object Destructuring

Just as arrays can be destructured, so too can objects.

Let's use the following object literal definition.

```
const photo = {
    id: 1,
    title: "Central Library",
    location: {
        country: "Canada",
        city: "Calgary"
    }
};
```

Object Destructuring (ii)

One can extract out a given property using dot or bracket notation as follows.

```
let id = photo.id;
let title = photo["title"];

let country = photo.location.country;
let city = photo.location["country"];
```

Equivalent assignments using object destructuring syntax would be:

```
let { id,title } = photo;
let { country,city } = photo.location;
```

These two statements could be combined into one:

```
let { id, title, location: {country,city} } = photo;
```

Spread

You can also make use of the spread syntax to copy contents from one array into another. Using the **photo** object from the previous section, we could copy some of its properties into another object using spread syntax:

```
const foo = { name:"Bob", .. .photo.location, iso:"CA" };
```

Is equivalent to:

```
const foo = { name:"Bob", country:"Canada", city:"Calgary", iso:"CA"};
```

It should be noted that this is a **shallow copy**, in that primitive values are copied,but for object references, only the references are copied.

JSON

JavaScript Object Notation or JSON is used as a language-independent data interchange format analogous in use to XML.

The main difference between JSON and object literal notation is that property names are enclosed in quotes, as shown in the following example:

```
// this is just a string though it looks like an object literal
const text = '{ "name1" : "value1",
   "name2" : "value2",
   "name3" : "value3"
}';
```

JSON object

The string literal on the last slide contains an object definition in JSON format (but is still just a string). To turn this string into an actual JavaScript object requires using the built-in JSON object.

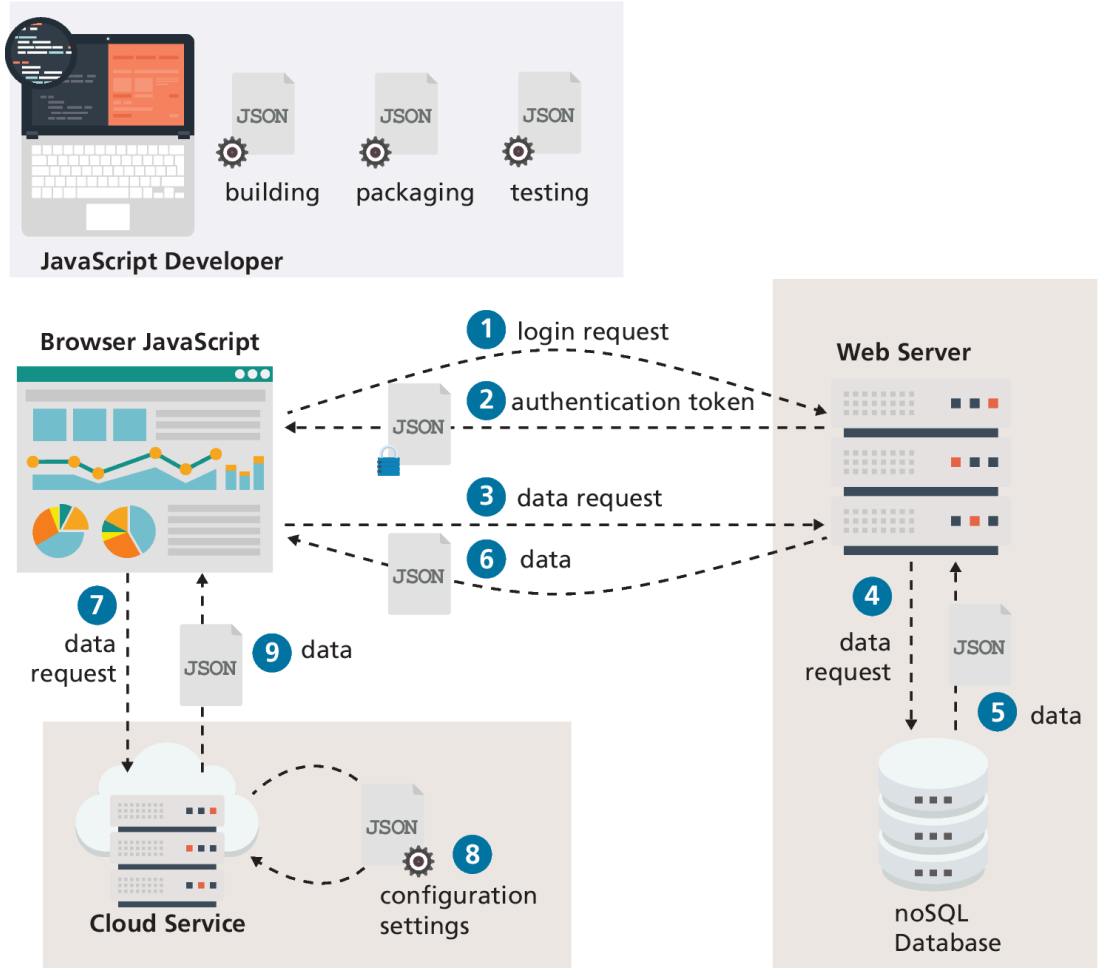
```
// this turns the JSON string into an object
const anObj = JSON.parse(text);
// displays "value1"
console.log(anObj.name1);
```

JSON in contemporary web development

JSON is encountered frequently in contemporary web development.

It is used by developers as part of their workflow, and most importantly, many web applications receive JSON from other sources, like other programs or websites.

This ability to interact with other web-based programs or sites will be covered in more detail in Chapter 10



Functions

Functions are defined by using the reserved word **function** and then the function name and (optional) parameters.

Functions do not require a return type, nor do the parameters require type specifications.

Declaring and calling functions

A function to calculate a subtotal as the price of a product multiplied by the quantity might be defined as follows:

```
function subtotal(price,quantity) {
  return price * quantity;
}
```

The above is formally called a **function declaration**. Such a declared function can be called or *invoked* by using the () operator.

```
let result = subtotal(10,2);
```

Function expressions

The object nature of functions can be further seen in the next example, which creates a function using a **function expression**.

When we invoked the function via the object variable name it is conventional to leave out the function name for so called **anonymous functions**

```
// defines a function using an anonymous function expression
const calculateSubtotal = function (price,quantity) {
                return price * quantity;
};

// invokes the function
let result = calculateSubtotal(10,2);

// define another function
const warn = function(msg) { alert(msg); };

// now invoke that function
warn("This doesn't return anything");
```

LISTING 8.11 Sample function expressions

Default Parameters

In the following code, what will happen (i.e., what will bar be equal to)?

```
function foo(a,b) {
  return a+b;
}
let bar = foo(3);
```

The answer is **NaN**. However, there is a way to specify **default parameters**

```
function foo(a=10,b=0) { return a+b; }
```

Now **bar** in the above example will be equal to 3.

Rest Parameters

How to write a function that can take a variable number of parameters?

The solution is to use the **rest operator** (...)

The concatenate method takes an indeterminate number of string parameters separated by spaces.

```
function concatenate(...args) {
        let s = "";
        for (let a of args)
                s += a + " ";
        return s;
}
let girls =
concatenate("fatima","hema","jane","alilah");
let boys = concatenate("jamal","nasir");

console.log(girls); //  "fatima hema jane alilah“

console.log(boys); // "jamal nasir“
```

Nested Functions

The object nature of functions can be further seen in the next example, which creates a function using a **function expression**.

When we invoked the function via the object variable name it is conventional to leave out the function name for so called **anonymous functions**

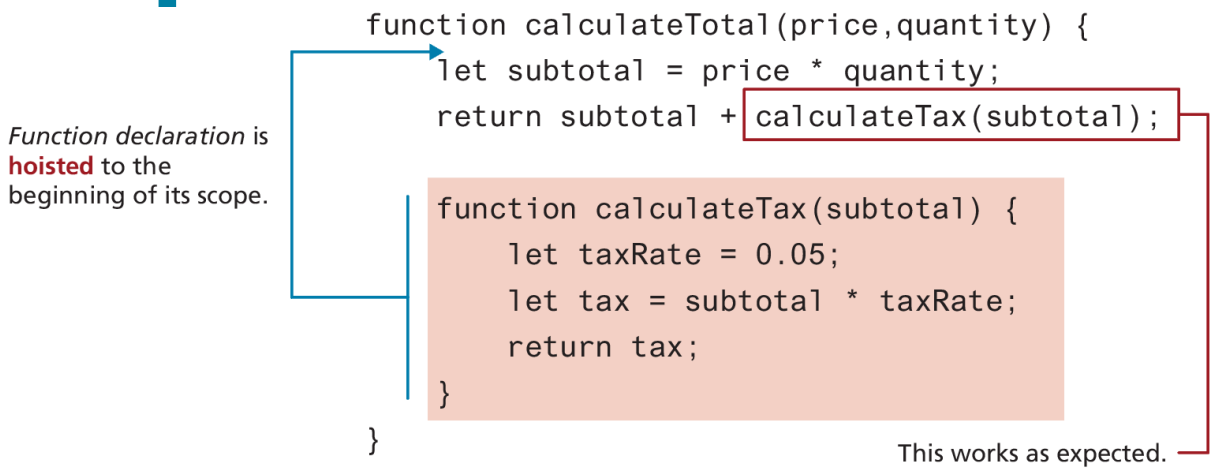
```
function calculateTotal(price,quantity) {
    let subtotal = price * quantity;
    return subtotal + calculateTax(subtotal);

    // this function is nested
    function calculateTax(subtotal) {
        let taxRate = 0.05;
        return subtotal * taxRate;
    }
}
```

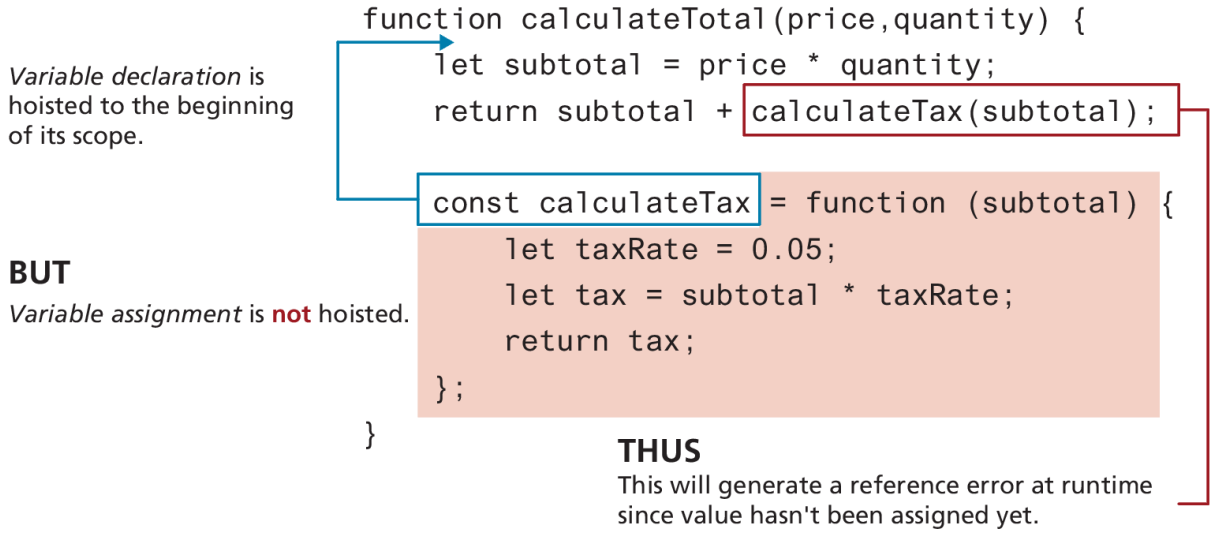
LISTING 8.12 Nesting functions

Hoisting in JavaScript

JavaScript function declarations are *hoisted* to the beginning of their current level

Note: the assignments are NOT hoisted.

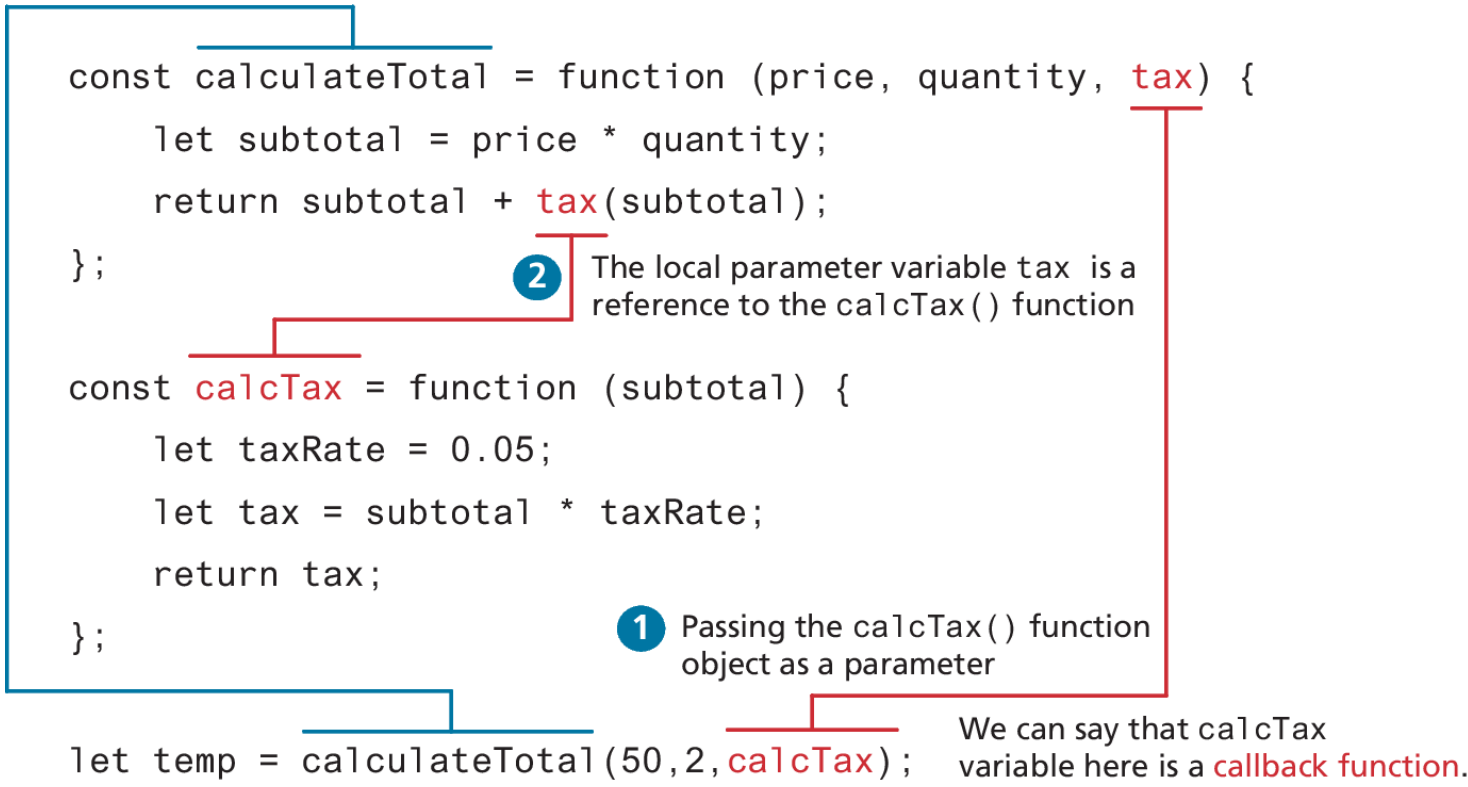




Callback Functions

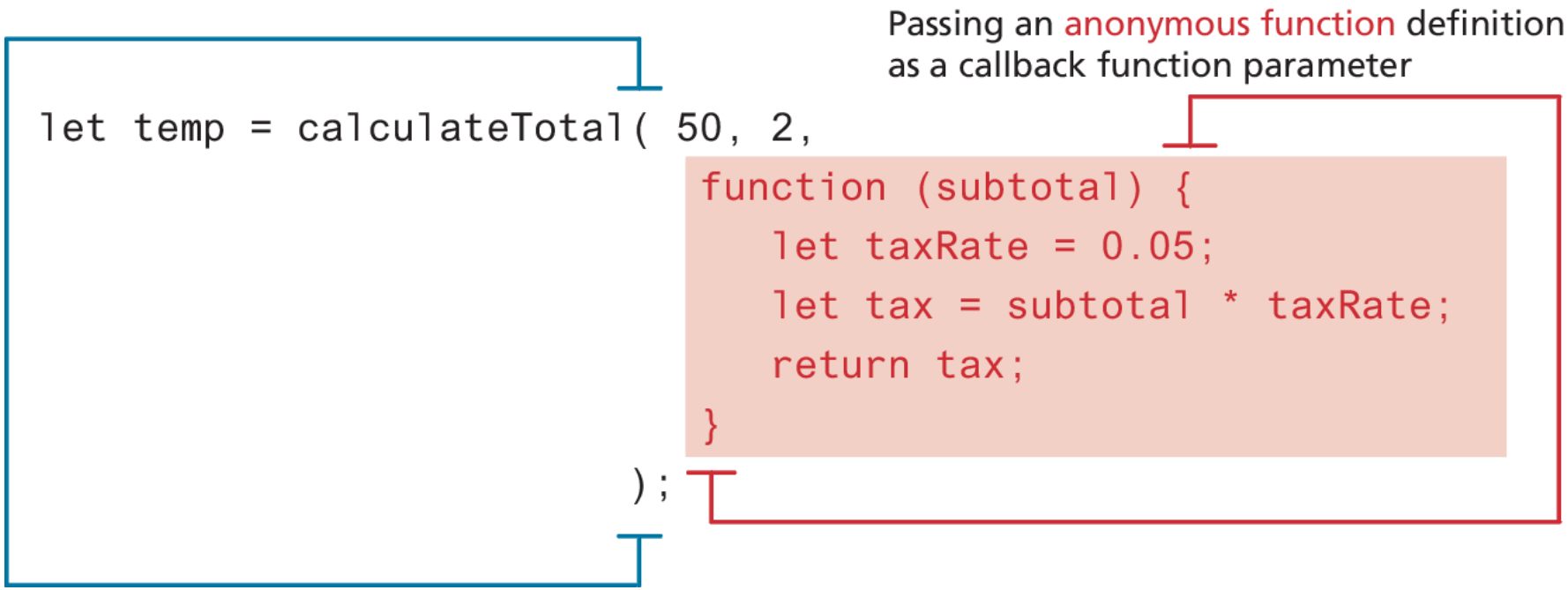
Since JavaScript functions are full-fledged objects, you can pass a function as an argument to another function.

Callback function is simply a function that is passed to another function.



Callback Functions (ii)

We can actually define a function definition directly within the invocation



Objects and Functions Together

In a class-oriented programming language like Java, we say classes define behavior via **methods**.

In a functional programming language like JavaScript, we say objects have properties that are **functions**.

Note the use of the keyword *this* in the two functions

```
const order ={
   salesDate : "May 5, 2016",
   product : {
      price: 500.00,
      brand: "Acer",
      output: function () {    return this.brand + ' $' + this.price; }
   },
   customer : {
      name: "Sue Smith",
      address: "123 Somewhere St",
      output: function () {return this.name + ', ' + this.address; }
   }
};

alert(order.product.output());
alert(order.customer.output());
```

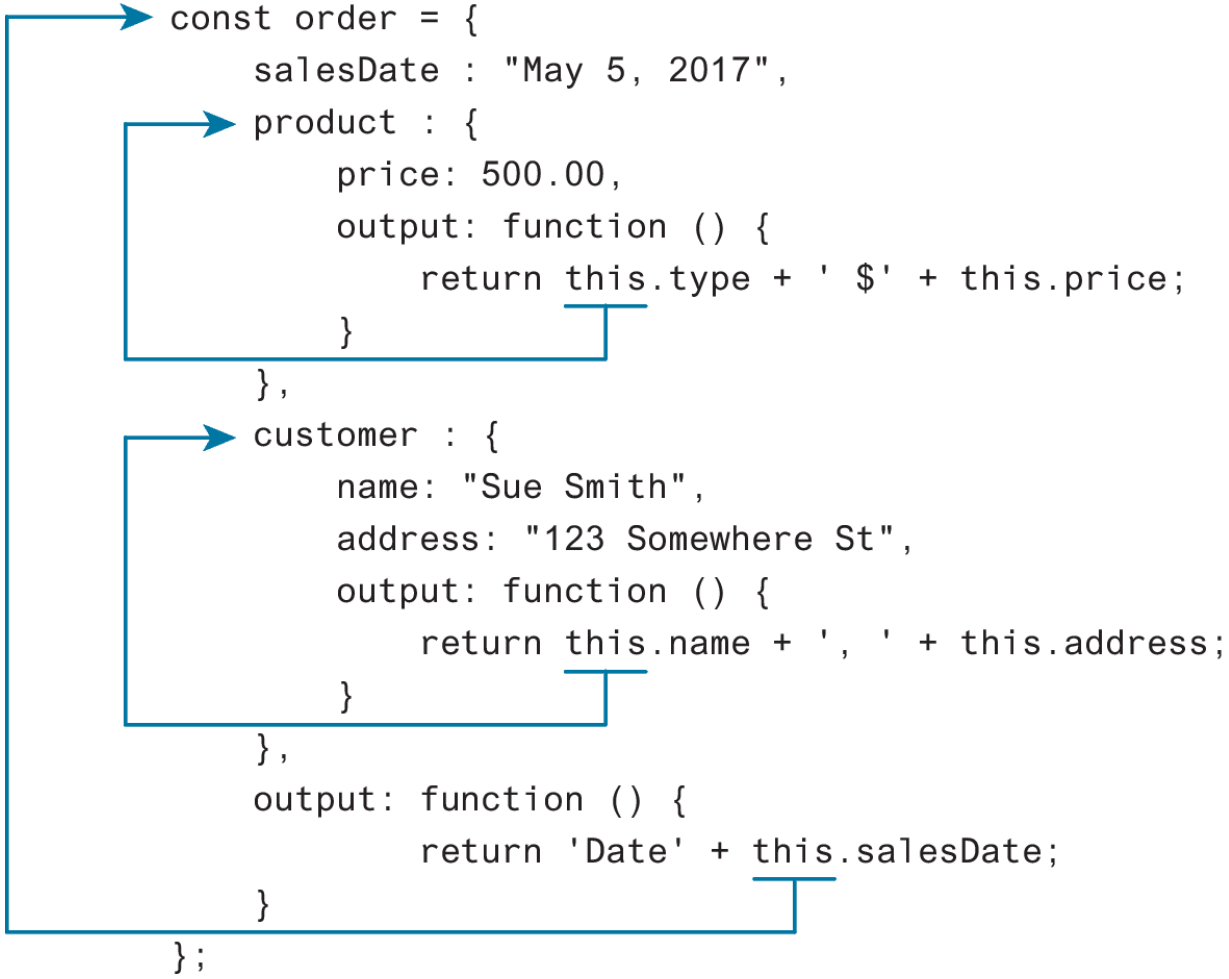
LISTING 8.13 Objects with functions

Contextual meaning of the this keyword

Note the use of the keyword *this* in the two functions

this is normally contextual and sometimes requires a full understanding of the current state

Luckily right now, we don't have to do anything so complex to understand the *this* in the example



Function constructors

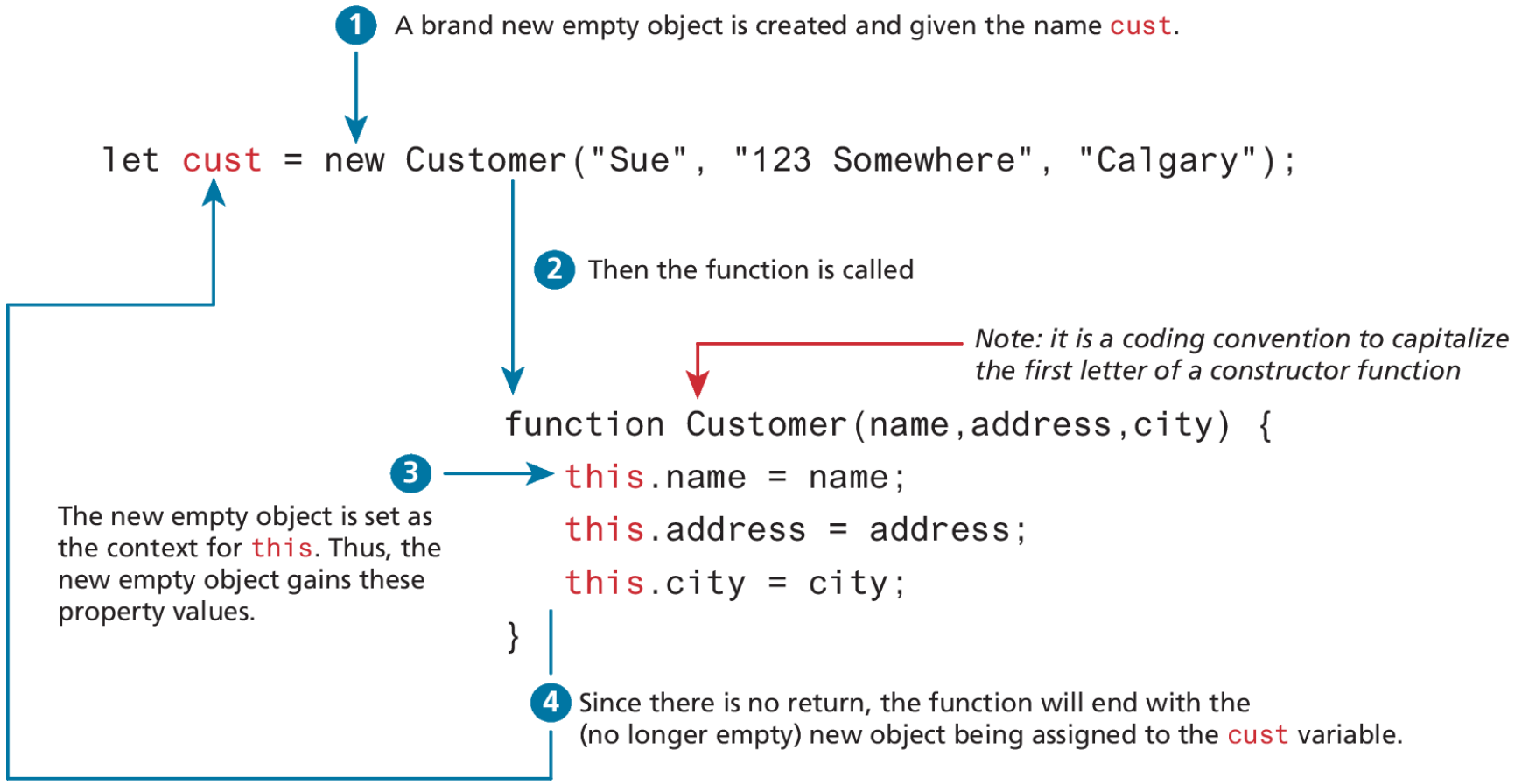
Looks similar to the approach used to create instances of objects in a class-based language like Java

The key difference between using a function constructor and using a regular function resides in the use of the **new** keyword before the function name.

```
// function constructor
function Customer(name,address,city) {
    this.name = name;
    this.address = address;
    this.city = city;
    this.output = function () {
        return this.name + " " + this.address + " " + this.city;
    };
}

// create instances of object using function constructor
const cust1 = new Customer("Sue", "123 Somewhere", "Calgary");
alert(cust1.output());
const cust2 = new Customer("Fred", "32 Nowhere St", "Seattle");
alert(cust2.output());
```

LISTING 8.14 Defining and using a function constructor

What happens with a constructor call of a function



Arrow Syntax

Arrow syntax provide a more concise syntax for the definition of anonymous functions. They also provide a solution to a potential scope problem encountered with the **this** keyword in callback functions.

To begin, consider an example of a simple function expression.

```
const taxRate = function () { return 0.05; };
```

The arrow function version would look like the following:

```
const taxRate = () => 0.05;
```

As you can see, this is a pretty concise (but perhaps confusing) way of writing code.

Array syntax overview

Traditional Syntax	Arrow Syntax	

`function () {`
`  statements`
`}`
	`() => {`
	`  statements`
	`}`
		Multi-line function,
		no parameters:
		{}, () required

`function (a,b) {`
`  statements`
`}`
	`(a,b) => {`
	`  statements`
	`}`
		Multi-line function,
		multiple parameters:
		() required

`function () {`
`  doSomething();`
`}`
	`() => {`
	`  doSomething();`
	`}`
		Single-line function,
		no return:
		{} required

`function (a) {`
`  return value;`
`}`
	`(a) => return value`	Single-line function,
		with return:
		{} optional

`function (a) {`
`  return value;`
`}`
	`a => value`	Single-line function,
		with return + one parameter:
		{}, (), return optional

Traditional Syntax	Arrow Syntax	

`function () {`
`  return value;`
`}`
	`() => value`	Single-line function,
		with return + no parameters:
		{}, return optional
		() required

`function (a,b) {`
`  return value;`
`}`
	`(a,b) => value`	Single-line function,
		with return + multiple parameters:
		{}, return optional
		() required

`const g = function(a) {`
`  return value;`
`}`
	`const g = a => value`	Function expression

`function (a,b) {`
`  return {`
`  p1: a,`
`  p2: b`
`  }`
`}`
	`(a,b) => ({`
	`  p1: a,`
	`  p2: b`
	`  })`
		When arrow function returns an object literal, the object literal must be wrapped in parentheses.

Changes to “this” in Arrow Functions

Arrow functions do not have their own **this** value (see Figure 8.22). Instead, the value of this within an arrow function is that of the enclosing lexical context (i.e., its enclosing parental scope at design time).

While this can occasionally be a limitation, it does allow the use of the **this** keyword in a way more familiar to object-oriented programming techniques in languages such as Java and C#.

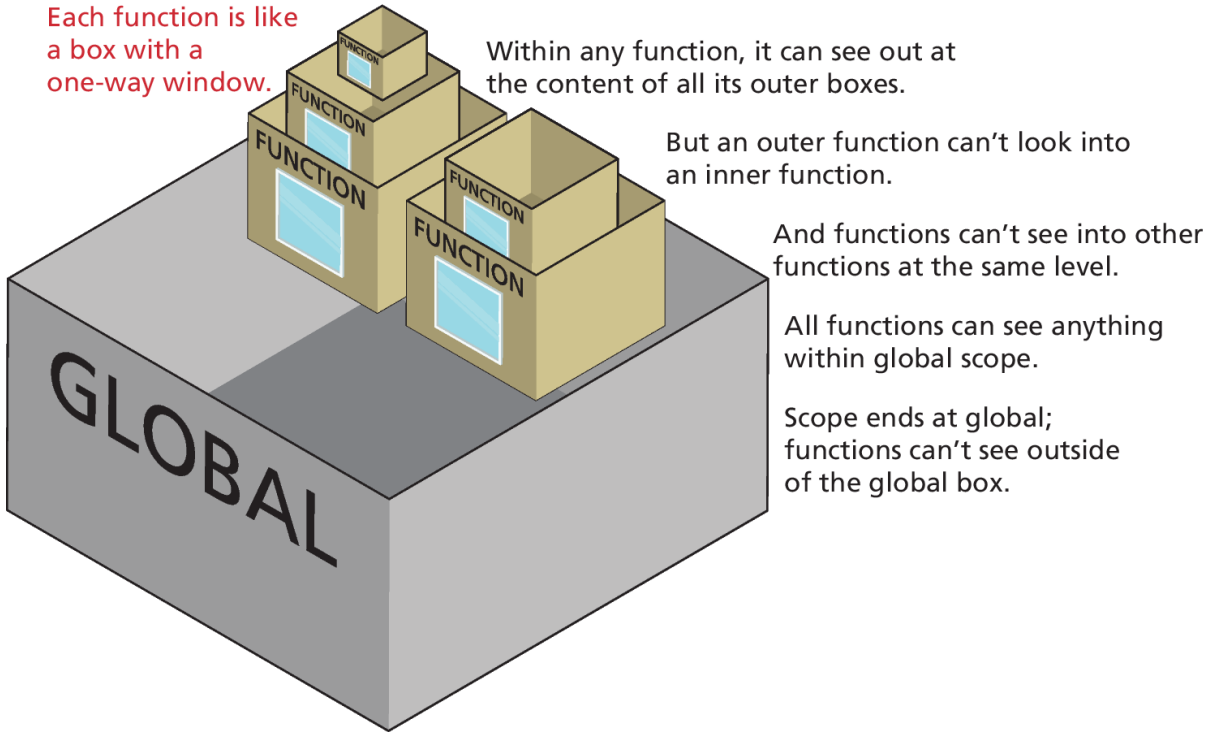
When we finally get to React development in Chapter 11, the use of arrow functions within ES6 classes will indeed make our code look a lot more like class code in a language like Java.

Scope in JavaScript

Scope generally refers to the context in which code is being executed.

JavaScript has four scopes:

- **function scope** (also called local scope),
- **block scope**,
- **module scope** (in Chapter 10)
- **global scope**.



Block scope

Block-level scope means variables defined within an **if {}** block or a **for {}** loop block using the **let** or **const** keywords are only available within the block in which they are defined. But if declared with **var** within a block, then it will be available outside the block.

3 Global Scope

```
for (var i=0; i<10;i++) {
   var tmp = "yes";
   console.log(tmp); outputs: yes
}
console.log(i);     outputs: 10
console.log(tmp);   outputs: yes
```

A variable will be in **global scope** if declared outside of a function *and* uses the `var` keyword.

4 Block Scope

```
for (let i=0; i<10;i++) {
   const tmp = "yes";
   console.log(tmp); outputs: yes
}
console.log(i);     error: i is not defined
console.log(tmp); error: tmp is not defined
```

A variable declared within a {} block using `let` or `const` will have **block scope** and *only* be available within the block it is defined.

Global Scope

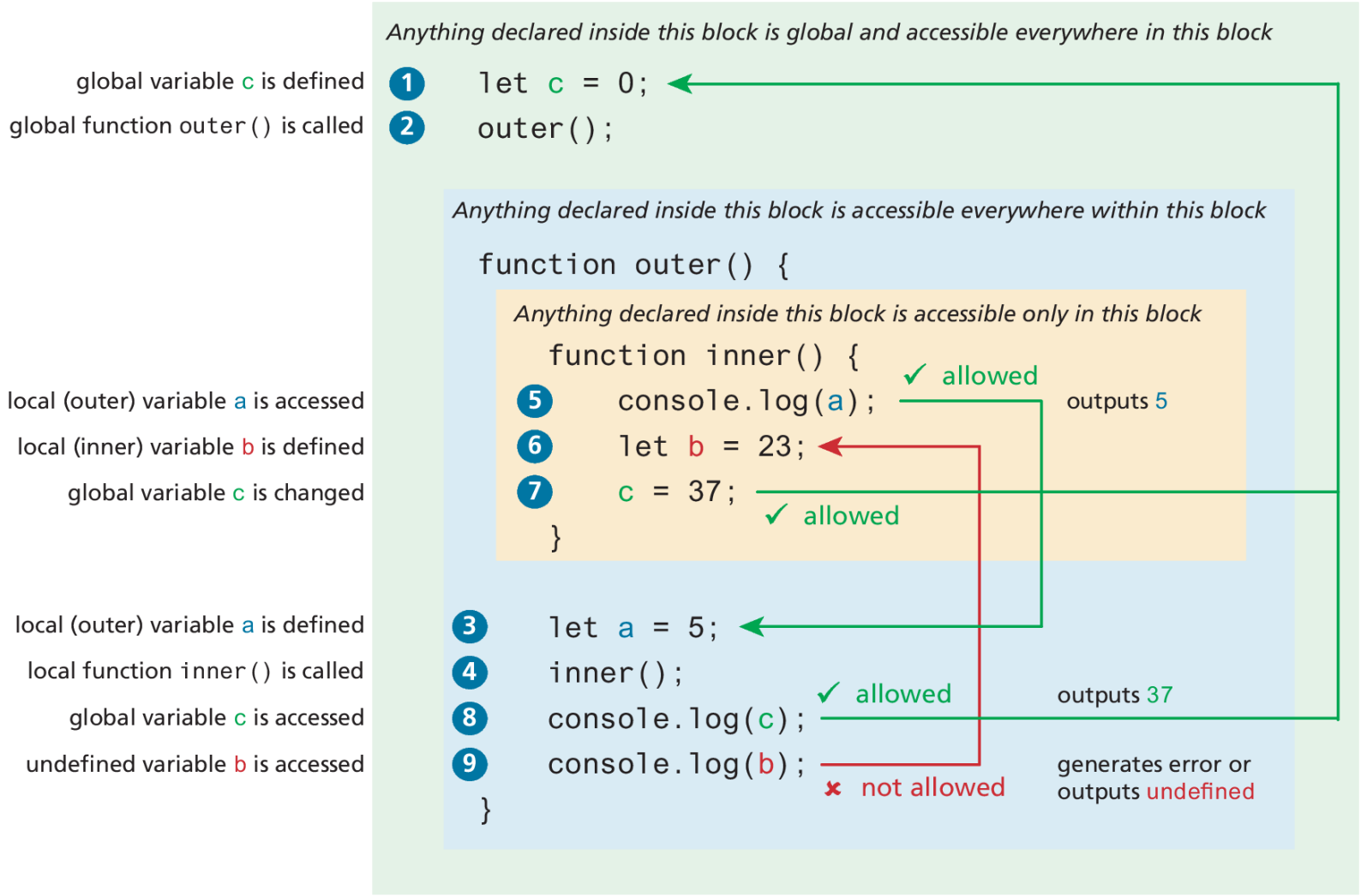
If an identifier has global scope, it is available everywhere.

Global scope can cause the **namespace conflict problem**.

- If the JavaScript compiler encounters another identifier with the same name at the same scope, you do not get an error. Instead, the new identifier *replaces* the old one!

In Chapter 10, you will learn of the new module feature in ES6 that helps address this problem.

Function/Local Scope



Closures in JavaScript

Scope in JavaScript is sometimes referred to as **lexical scope**

The ending bracket of a function is said to close the scope of that function. But closure refers to more than just this idea.

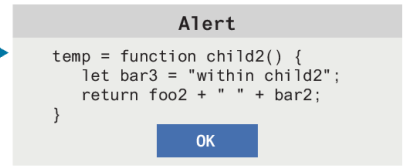
A **closure** is actually an object consisting of the scope environment in which the function is created; that is, a closure is a function that has an implicitly permanent link between itself and its scope chain.

```
let g2 = "variable with global scope";

function parent2() {
   let foo2 = "within parent2";

   function child2() {
      let bar2 = "within child2";
      return foo2 + " " + bar2;
   }

   return child2;
}

let temp = parent2();
alert("temp = " + temp);

alert("temp() = " + temp());

console.dir(temp);
```

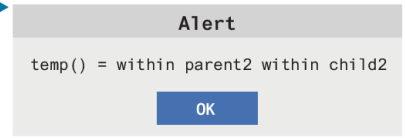
Notice that we are **not** invoking the inner function now.

Instead, we are returning the inner function (and not its return value as in previous example).

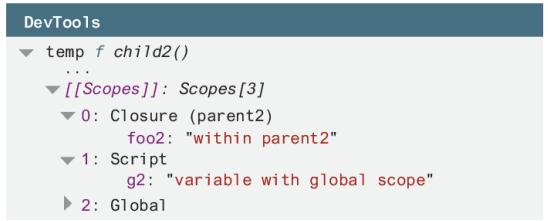
The temp variable is now going to contain inner child2() function.



The temp function still has access to the foo2 variable within the parent2 function even though the temp function is now outside its declared lexical scope (i.e., the parent2 function).



It has this same access since the closure keeps a record of the lexical (design-time) scope environment.



After parent2 executes, we might expect that any local variables defined within the function to be gone (i.e., garbage collected).

Yet in this example, this is not what happens. The local variable foo2 sticks around even after parent2 is finished executing. Why?

This happens because the parent2 function has a **closure**.

A **closure** is like a special object that contains a function's design-time scope environment. A closure thus lets a function continue to access its design-time lexical scope even if it is executed outside its original parent.

Key Terms

AJAX

anonymous functions

array literal notation

assignment

arrays

arrow syntax

block scope

browser extension

browser plug-in

built-in objects

callback function

client-side scripting

closure

conditional operator

default parameters

dot notation

dynamically typed

ECMAScript

ES6

exception

falsy

for loops

functions

function constructor

function declaration

function expression

function scope

global scope

JavaScript frameworks

JavaScript Object Notation

JSON

keyword

lexical scope

loop control variable

method

module scope

namespace conflict

problem

objects

object literal notation

primitive types

property

reference types

rest operator

scope (local and global)

shallow copy

spread syntax

template literals

ternary operator

truthy

try. . . catch block

undefined

variables

Fundamentals of Web Development

Third Edition by Randy Connolly and Ricardo Hoar

Chapter 9

JavaScript 2:

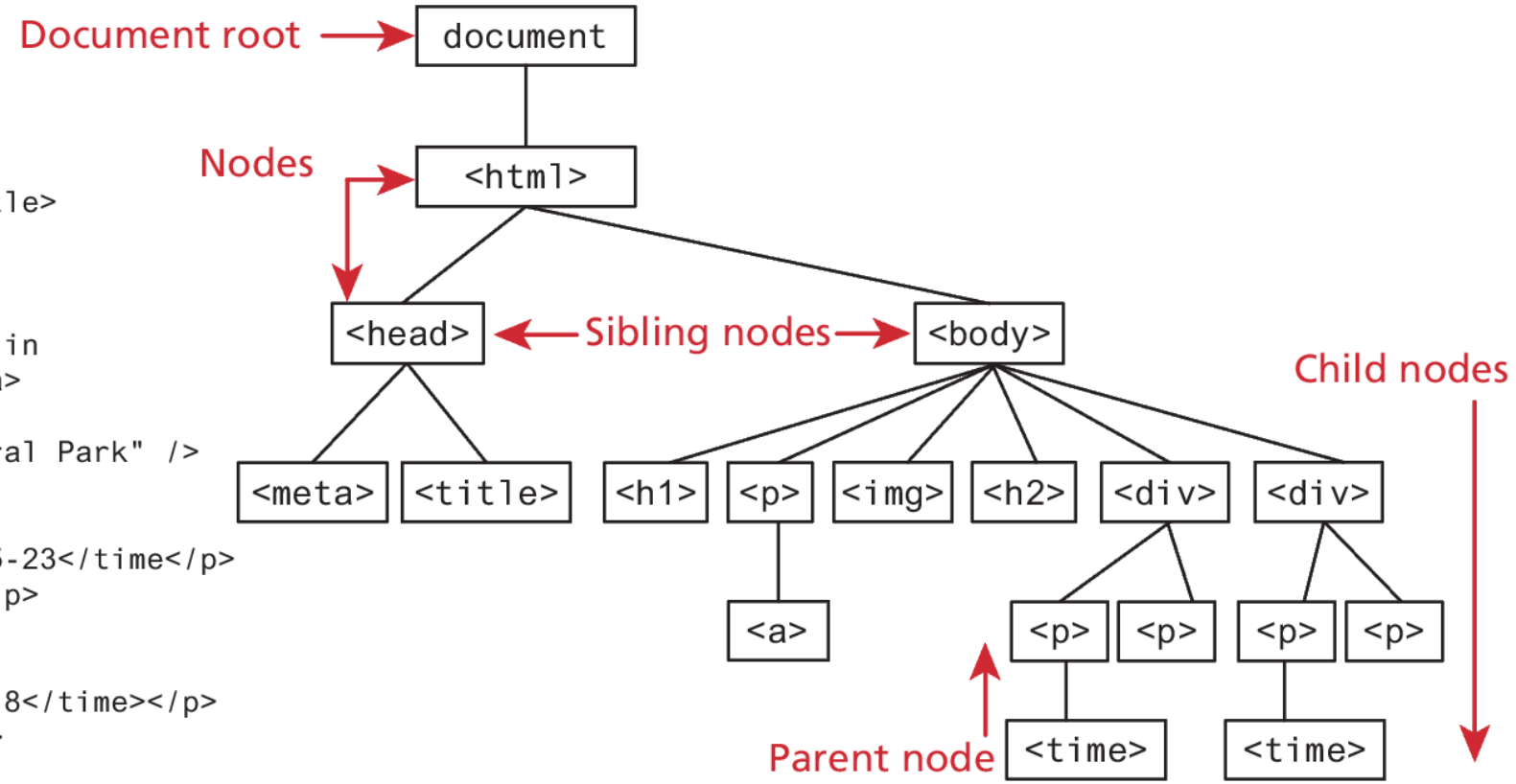
Using JavaScript

In this chapter you will learn . . .

- What is Document Object Model (DOM)
- How to use the DOM to dynamically manipulate the contents of a web page
- How to use the DOM and event handling to validate user input in a form
- What are regular expressions and how to use them in JavaScript.

The Document Object Model (DOM)

```
<html>
<head>
   <meta charset="utf-8">
   <title>Share Your Travels</title>
</head>
<body>
   <h1>Share Your Travels</h1>
   <p>Photo of Conservatory Pond in
      <a href="#">Central Park</a>
   </p>
   <img src="park.jpg" alt="Central Park" />
   <h2>Reviews</h2>
   <div id="latestComment">
      <p>Ricardo on <time>2021-05-23</time></p>
      <p>Easy on the HDR buddy.</p>
   </div>
   <div>
      <p>Susan on <time>2021-11-18</time></p>
      <p>I love Central Park.</p>
   </div>
</body>
</html>
```

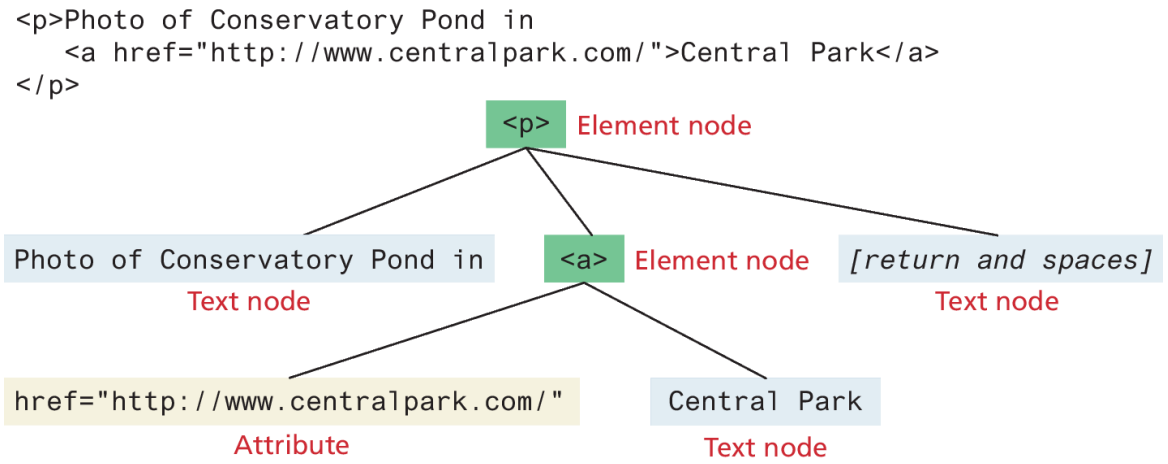


DOM Nodes and NodeLists

In the DOM, each element within the HTML document is called a **node**.

The DOM also defines a specialized object called a **NodeList** that represents a collection of nodes. It operates very similarly to an array.

Many programming tasks that we typically perform in JavaScript involve finding one or more nodes and then modifying them.



Some Essential Node Object Properties

- **childNodes** A NodeList of child nodes for this node
- **firstChild** First child node of this node
- **lastChild** Last child of this node
- **nextSibling** Next sibling node for this node
- **nodeName** Name of the node
- **nodeType** Type of the node
- **nodeValue** Value of the node
- **parentNode** Parent node for this node
- **previousSibling** Previous sibling node for this node
- **textContent** Represents the text content (stripped of any tags) of the node

Document Object

The **DOM document object** is the root JavaScript object representing the entire HTML document. It is globally accessible via the **document** object reference.

The properties of a document cover information about the page. Some are read-only, but others are modifiable. Like any JavaScript object, you can access its properties using either dot notation or square bracket notation

```
// retrieve the URL of the current page
let a = document.URL;
// retrieve the page encoding, for example ISO-8859-1
let b = document["inputEncoding"];
```

Document Methods

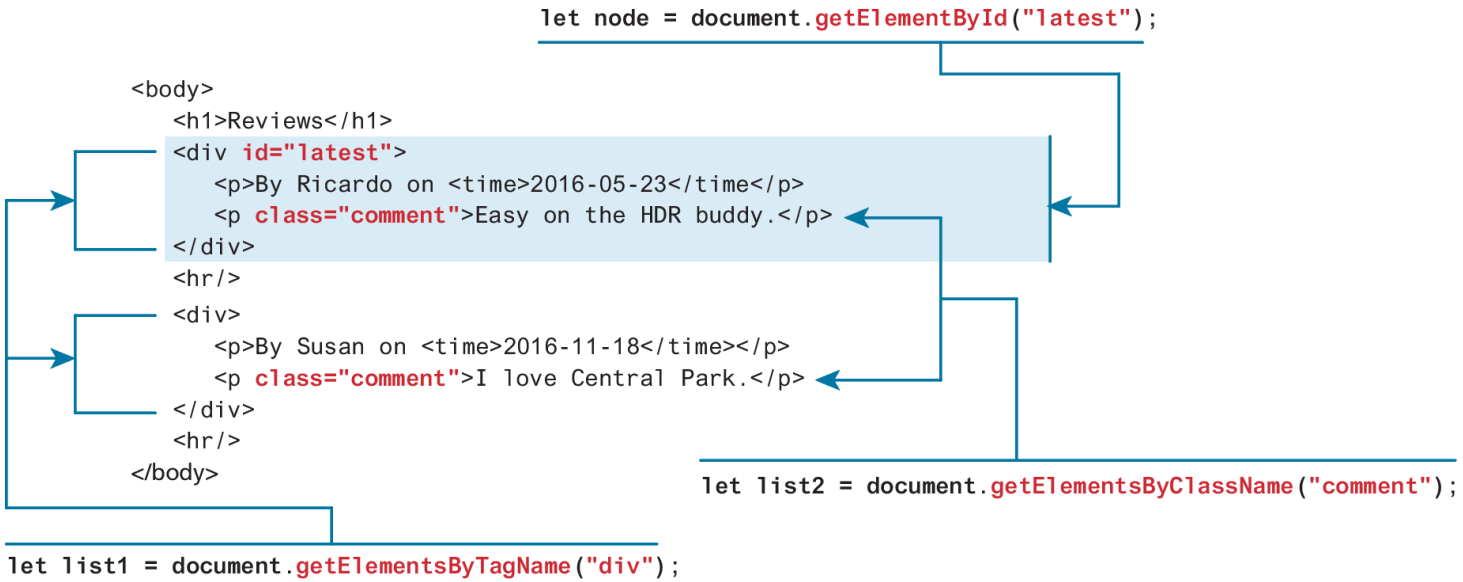
In addition to these properties, there are several essential methods you will use all the time (We used **document.write(**) last chapter). These methods fall into three categories

- Selection methods
- Family manipulation methods
- Event methods

Selection Methods

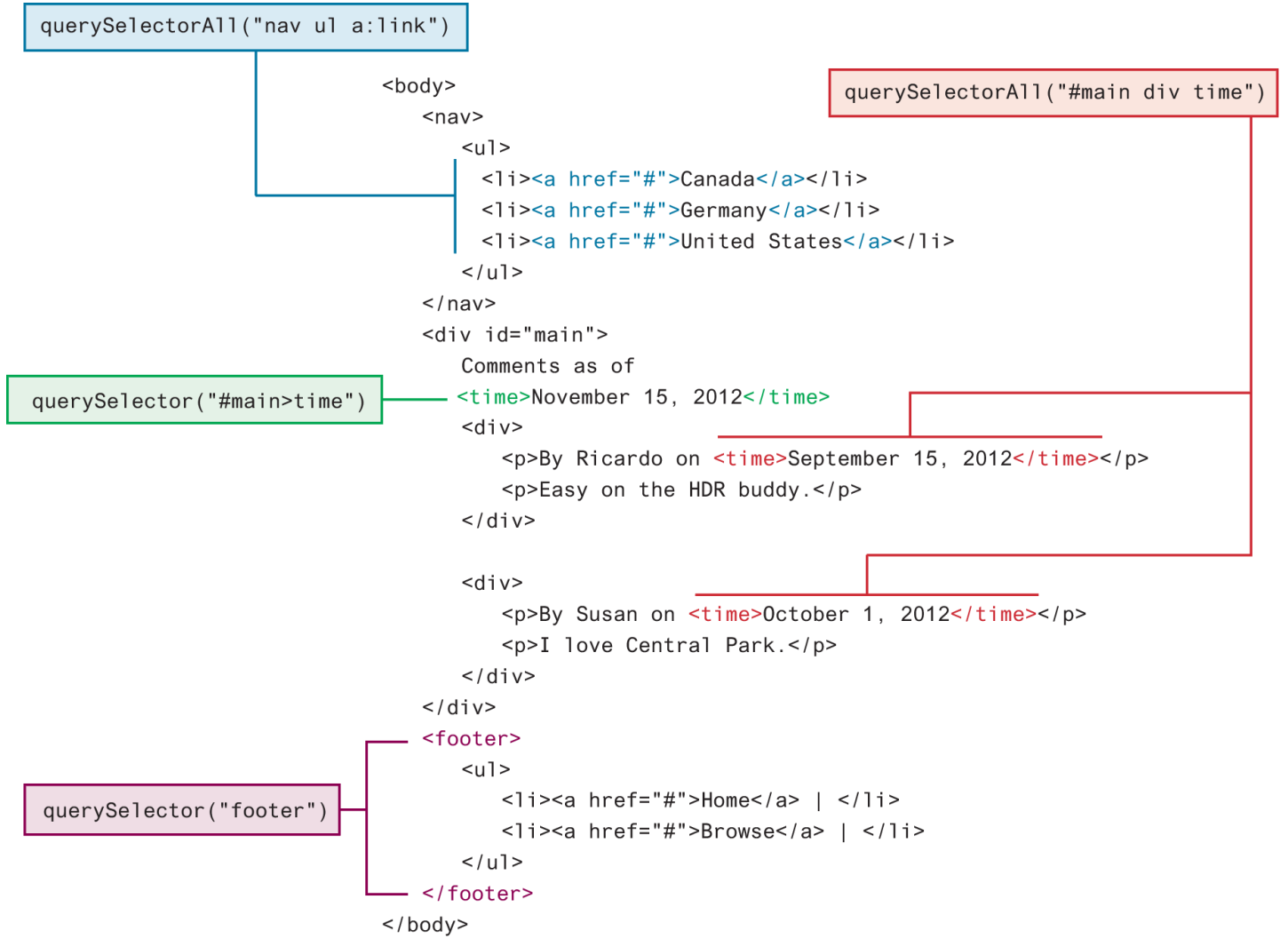
The most important DOM methods

They allow you to select one or more document elements. The oldest 3 are: **getElementById("*id*"), getElementsByClassName("*name*")** and **getElementsByTagName("*name*")**



Query Selection Methods

The newer **querySelector**() and **querySelectorAll**() methods allow you to query for DOM elements much the same way you specify CSS styles



Element Node Object

Element Node object represents an HTML element in the hierarchy, contained between the opening <> and closing </>.

An element can itself contain more elements

Every element node has the node properties shown in Table 9.1 (slide 5)

It also has a variety of additional properties, the most important of which are shown in Table 9.3 (next slide)

Some Essential Element Node Properties

- **classList** A read-only list of CSS classes assigned to this element. This list has a variety of helper methods for manipulating this list.
- **className** The current value for the class attribute of this HTML element.
- **id** The current value for the id of this element.
- **innerHTML** Represents all the content (text and tags) of the element.
- **style** The style attribute of an element. This returns a CSSStyleDeclaration object that contains sub-properties that correspond to the various CSS properties.
- **tagName** The tag name for the element.

Extra Properties for Certain Tag Types

Property	Description	Tags
href	Used in <a> tags to specify the linking URL.	a
name	Used to identify a tag. Unlike id which is available to all tags, name is limited to certain form-related tags.	a, input, textarea, form
src	Links to an external URL that should be loaded into the page (as opposed to href which is a link to follow when clicked).	img, input, iframe, script
value	Provides access to the value attribute of input tags. Typically used to access the user's input into a form field.	input, textarea, submit

TABLE 9.4 Some Specific HTML DOM Element Properties for Certain Tag Types

Accessing elements and their properties

```
<p id="here">hello <span>there</span></p>
<ul>
    <li>France</li>
    <li>Spain</li>
    <li>Thailand</li>
</ul>
<div id="main">
    <a href="somewhere.html">
        <img src="whatever.gif" class="thumb">
    </a>
</div>

<script>

const node = document.getElementById("here");
console.log(node.innerHTML); // hello <span>there</span>
console.log(node.textContent); //“hello there”

const items = document.getElementsByTagName("li");
for (let i=0; i<items.length; i++) {
    // outputs: France, then Spain, then Thailand
    console.log(items[i].textContent);
}

const link = document.querySelector("#main a");

console.log(link.href); // outputs: somewhere.html

const img = document.querySelector("#main img");
console.log(img.src); // outputs: whatever.gif

console.log(img.className); // outputs: thumb

</script>
```

LISTING 9.1 Accessing elements and their properties

Modifying the DOM

Now that you can access some of the node and element properties you might be wondering how one can make use of some of these properties. Since most of the properties listed in the previous tables are all read and write, this means that they can be programmatically changed.

- Changing an Element's Style
- Changing the content of any given element
- DOM Manipulation Methods

Changing an Element's Style

To programmatically modify the styles associated with a particular element one must change the properties of the style property for that element

For instance, to change an element's background color and add a three pixel border, we could use the following code:

```
const node = document.getElementById("someId");
node.style.backgroundColor = "#FFFF00";
node.style.borderWidth = "3px";
```

How CSS styles can be programmatically manipulated in JavaScript

While you can directly change CSS style elements via this **style** property, it is generally preferable to change the appearance of an element instead using the **className** or **classList** properties

```
<style>
    .box {
        margin: 2em; padding: 0;
        border: solid 1pt black;
    }
   .yellowish { background-color: #EFE63F; }
   .hide { display: none; }
</style>
<main>
   <div class="box">
    ...
   </div>
</main>
```

```
var node = document.querySelector("main div");

1 node.className = "yellowish";
    → This replaces the existing class specification with this one. Thus the <div> no longer has the box class

2 node.classList.remove("yellowish");
  node.classList.add("box");
    → Removes the specified class specification and adds the box class

3 node.classList.add("yellowish");
    → Adds a new class to the existing class specification

4 node.classList.toggle("hide");
    → If it isn't in the class specification, then add it

5 node.classList.toggle("hide");
    → If it is in the class specification, then remove it
```

Equivalent to:

```
1 <div class="yellowish">

2 <div class="">
  <div class="box">

3 <div class="box yellowish">

4 <div class="box yellowish hide">

5 <div class="box yellowish">
```

InnerHTML vs textContent vs DOM Manipulation

Listing 9.1 (slide 13) illustrated how you can programmatically access the content of an element node through its innerHTML or textContent property. These properties can also be used to modify the content of any given element.

For instance, you could change the content of the in Listing 9.1 using the following:

```
const div = document.querySelector("#main");
div.innerHTML = '<a href="#"><img src="ab.gif"></a>';
```

This replaces the existing content with the new content.

InnerHTML vs textContent vs DOM Manipulation (ii)

Using **innerHTML** is generally discouraged (even though you will likely see many examples online that use these approaches) because they are potentially vulnerable to Cross-Site Scripting (XSS) attacks

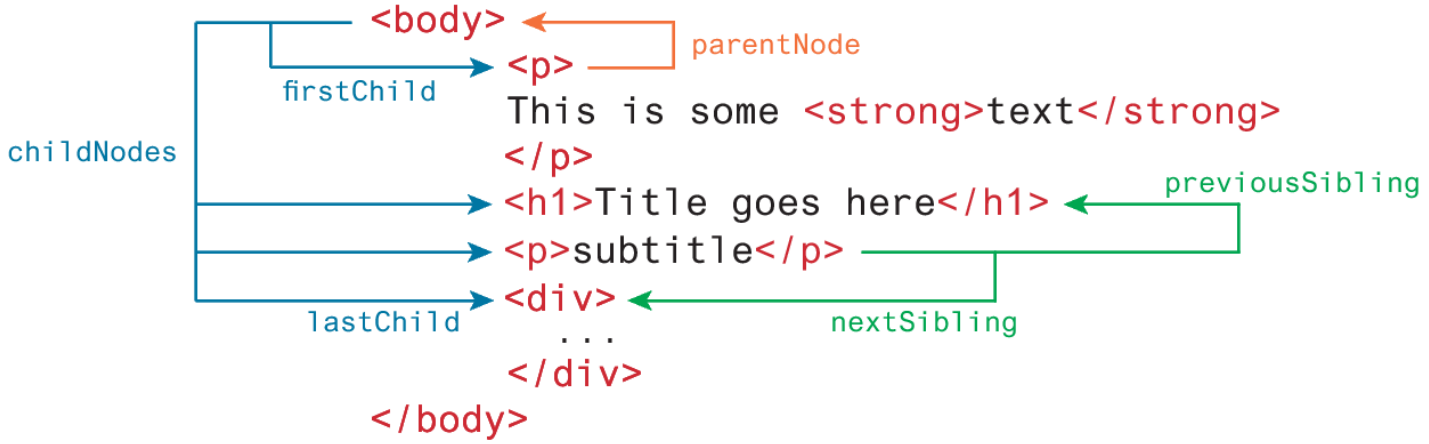
In practice, when you need to change the inner text of an element, it is preferable to use the **textContent** property instead of **innerHTML** since any markup is stripped from it.

In addition, when you need to generate HTML elements, it is better to use the appropriate DOM manipulation methods covered in the next section

DOM family relations

Each node in the DOM has a variety of “family relations” properties and methods for navigating between elements and for adding or removing elements from the document hierarchy.

Child and sibling properties can be an unreliable mechanism for selecting nodes and thus, in general, you will instead use selector methods



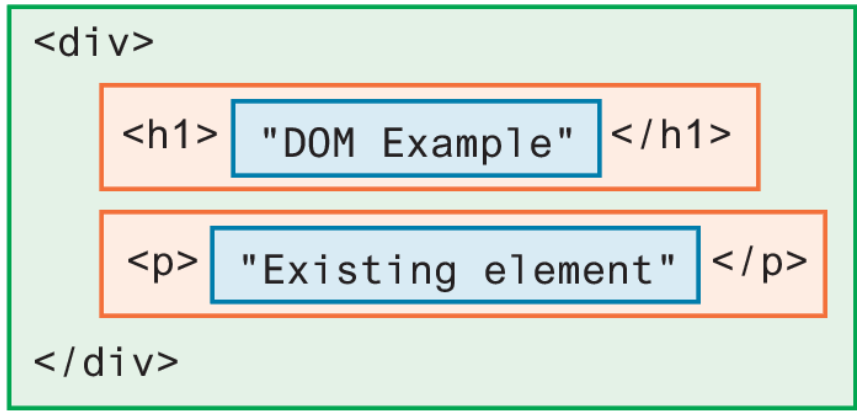
DOM Manipulation Methods

- **appendChild** Adds a new child node to the end of the current node.
- **createAttribute** Creates a new attribute node.
- **createElement** Creates an HTML element node.
- **createTextNode** Creates a text node.
- **insertAdjacentElement** Inserts a new child node at one of four positions relative to the current node.
- **insertAdjacentText** Inserts a new text node at one of four positions relative to the current node.
- **insertBefore** Inserts a new child node before a reference node in the current node.
- **removeChild** Removes a child from the current node.
- **replaceChild** Replaces a child node with a different child.

Visualizing the DOM modification

```
<div id="first">
    <h1>DOM Example</h1>
    <p>Existing element</p>
</div>
```

Visualizing the DOM elements



1. Create a new text node

"this is dynamic"

```
const text = document.createTextNode("this is dynamic");
```

2. Create a new empty element

```
const p = document.createElement("p");
```

Visualizing the DOM modification (ii)

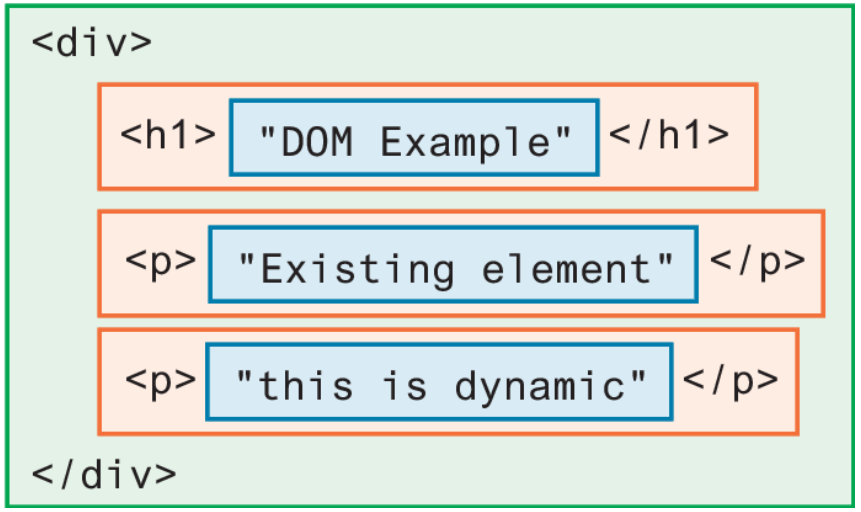
3 Add the text node to new <p> element

```
p.appendChild(text);
```

4 Add the <p> element to the

```
const first = document.getElementById("first");
first.appendChild(p);
```

```
<div id="first">
    <h1>DOM Example</h1>
    <p>Existing element</p>
    <p>this is dynamic</p>
</div>
```



DOM Timing

Before finishing this section on using the DOM, it should be emphasized that the timing of any DOM code is very important.

You cannot access or modify the DOM until it has been loaded.

If the DOM programming is written *after* the markup as in Listing 9.2 that *should* ensure that the elements exist in the DOM before the code executes.

To wait until we know for sure that the DOM has been loaded requires knowledge from our next section on **event handling**.

JavaScript Event Handling

Implementing an Event Handler

An event handler is first defined, then registered to an element node object.

Registering an event handler requires passing a callback function to the **addEventListener()**

```
<input type="submit" id="btn">
<script>
  function simpleHandler() {
     alert("button was clicked");
  }
  const btn = document.querySelector("#btn");
  btn.addEventListener("click", simpleHandler);
</script>
```

1 Event handler defined

2 Event handler registered

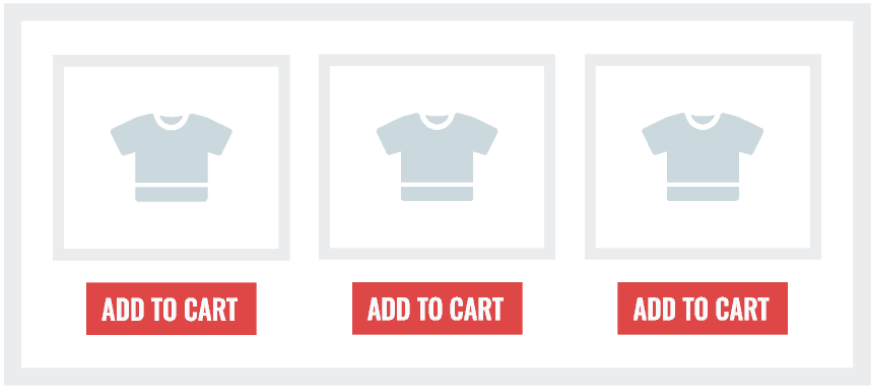
Listening with an anonymous function

It is much more common to make use of an *anonymous function* passed to **addEventListener**()

```
const btn = document.getElementById("btn");
    btn.addEventListener("click", function () {
    alert("used an anonymous function");
});

document.querySelector("#btn").addEventListener("click", function (){
    alert("a different approach but same result");
});

document.querySelector("#btn").addEventListener("click", () => {
    alert("arrow syntax but same result");
});
```

LISTING 9.3 Listening to an event with an anonymous function, three versions

Event handling with NodeList arrays




```
<ul id="list">
  <li>
    <img src="a.png" ... >
    <button>Add To Cart</button>
  </li>
  <li>
    <img src="b.png" ... >
    <button>Add To Cart</button>
  </li>
  <li>
    <img src="c.png" ... >
    <button>Add To Cart</button>
  </li>
</ul>
```

```
// select all the buttons
const btns = document.querySelectorAll("#list button");

// this won't work and will generate error
btns.addEventListener("click", function () { ... });
```

```
// instead must loop through node list ...
for (let bt of btns) {
   // ...and assign event listener to each node
   bt.addEventListener("click", function () { ... });
}
```

Remember that a node list (i.e., array of nodes) doesn't support event listeners. Only individual node objects have the `addEventListener()` method defined.

Page Loading and the DOM

To ensure your DOM manipulation code *after* the page is loaded use one of the following two different page load events.

- **window.load** Fires when the entire page is loaded. This includes images and stylesheets, so on a slow connection or a page with a lot of images, the load event can take a long time to fire.

- **document.DOMContentLoaded** Fires when the HTML document has been completely downloaded and parsed. Generally, this is the event you want to use.

Using one of these, your DOM coding can now appear anywhere, including within the **<head>** element, which is the conventional place to add in your JavaScript code.

Wrapping DOM code within a DOMContentLoaded event handler

```
document.addEventListener('DOMContentLoaded', function() {
   const menu = document.querySelectorAll("#menu li");
   for (let item of menu) {
      item.addEventListener("click", function () {
         item.classList.toggle('shadow’);
      });
   }
   const heading = document.querySelector("h3");
   heading.addEventListener('click', function() {
      heading.classList.toggle('shadow’);
   });
});
```

LISTING 9.4 Wrapping DOM code within a DOMContentLoaded event handler

Event Object

- When an event is triggered, the browser will construct an **event object** that contains information about the event.
- Your event handlers can access this event object simply by including it as an argument to the callback function (by convention, this event object parameter is often named *e*)

Event Object Example


```
<ul id="menu">
  <li>Home</li>
  <li>About</li>
  <li>Products</li>
  <li>Contact</li>
</ul>
```

```
const menu = document.querySelectorAll("#menu li");
for (let item of menu) {
  item.addEventListener("click", menuHandler );
}
function menuHandler(e) {
    const x = e.clientX;
    const y = e.clientY;
    displayArrow(x,y);
    e.target.classList.toggle("selected");
    performMenuAction(e.target.innerHTML);
}
```

By receiving the event object as a parameter and using it to reference the clicked item, the `menuHandler()` function will work no matter where it is located.

Click events include the on-screen pixel location of the mouse cursor.

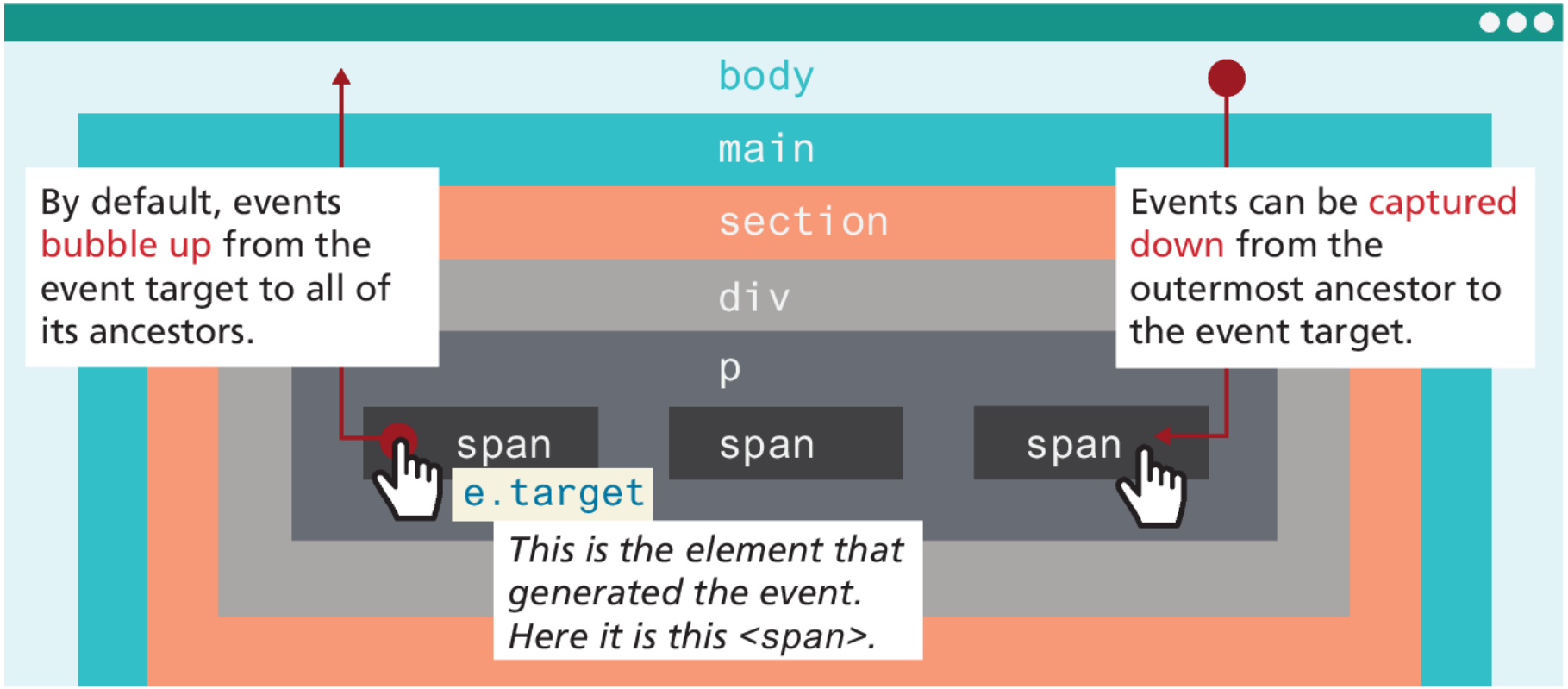
The `e.target` object in this case is referencing the clicked `<li>` item.

Event Propagation

When an event fires on an element that has ancestor elements, the event propagates to those ancestors. There are two distinct phases of propagation:

- In the **event capturing phase**, the browser checks the outermost ancestor (the <html> element) to see if that element has an event handler registered for the triggered event, and if so, it is executed. It then proceeds to the next ancestor and performs the same steps; this continues until it reaches the element that triggered the event (that is, the **event target**).
- In the **event bubbling phase**, the opposite occurs. The browser checks if the element that triggered the event has an event handler registered for that event, and if so, it is executed.

Event capture and bubbling



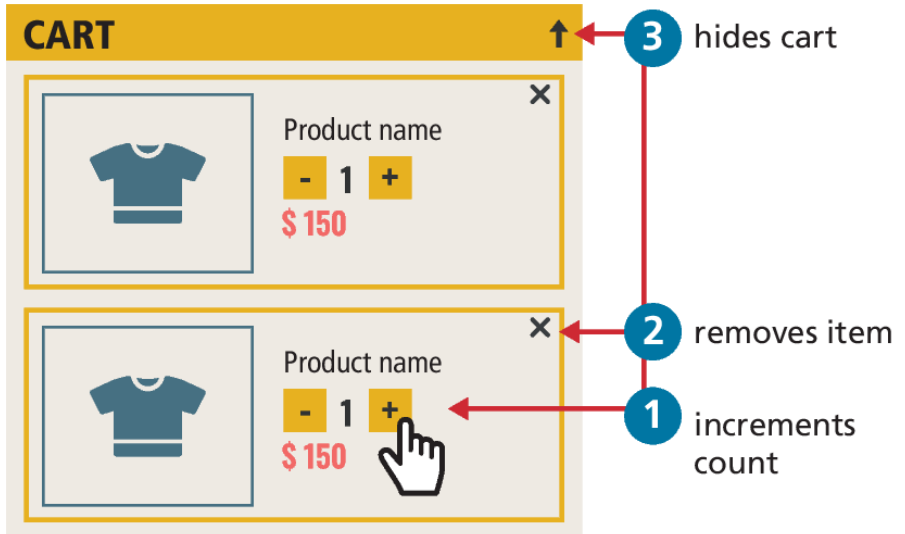
`e.currentTarget` *This is the element whose event handler is currently being executed. In this example, it could the or any of its ancestors.*

Problems with event propagation

Occasionally, the bubbling of events can cause problems. For instance consider elements nested within one another, each with its own on-click behaviors.

When the user clicks on the increment count button, the click handler for the increment <button> will trigger first. Unfortunately, it will then trigger the click event for the , and the <aside> element!

Thankfully, there is a solution to such problems. The stopPropagation() method of the event argument object will stop event propagation.




```
<aside id="cart">
  <h2>Cart</h2>
  <div class="item">
    <h3>Product Name</h3>
    ...
    <button class="plus">
    ...
</aside>
```

Stopping event propagation

```
const btns = document.querySelectorAll(".plus");
for (let b of btns) {
   b.addEventListener("click", function (e) {
      e.stopPropagation();
      incrementCount(e);
   });
}

const items = document.querySelectorAll(".item");
for (let it of items) {
   it.addEventListener("click", function (e) {
      e.stopPropagation();
      removeItemFromCart(e);
   });
}
```

```
const aside = document.querySelector("aside#cart");
aside.addEventListener("click", function () {
      minimizeCart();
});
```

LISTING 9.5 Stopping event propagation

Event Delegation

To avoid creating duplicate event handlers for each element within a **NodeList**, an alternative is to use **event delegation** where we assign a single listener to the parent and make use of event bubbling

Suppose we have numerous image thumbnails within a parent element, similar to the following:

```
<body>
<header>...</header>
<main>
<section id="list">
<h2>Section Title</h2>
<img ... />
<img ... />
...
</section>
</main>
</body>
```

Event Delegation (ii)

Now what if you wanted to do something special when the user clicks the mouse on an <img>

You would probably write something like the following:

Notice that this solution adds an event listener to every <img> element.

```
const images =   document.querySelectorAll("#list img");
for (let img of images) {
        img.addEventListener("click", someHandler);
}
```

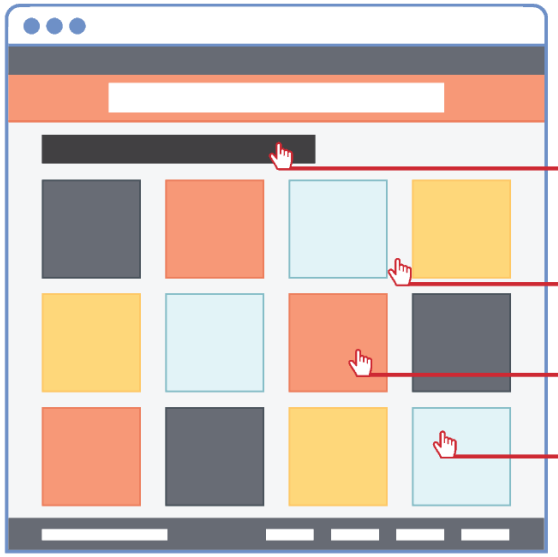
Event Delegation (iii)

Instead, we can add a single listener to the parent element, as shown in the following code

Since the user can click on all elements within the <section> element (as can be seen in Figure 9.15), the click event handler needs to determine if the user has clicked on one of the <img> elements within it.

```
const parent = document.querySelector("#list");
 parent.addEventListener("click", function (e) {
 // e.target is the object that generated the event.
 // to verify that e.target exists and that it is one of the
 // <img> elements. Note: NodeName always returns
 //upper case
 if (e.target && e.target.nodeName == "IMG") {
    doSomething(e.target);
 }
});
```

Event Delegation (iv)



These clicks are within the <section>, so they will trigger our event. We want to ignore these click events.

These clicks are within the <section>, so they will trigger our event. We only want to process these click events because they are within an <img>.

```
<body>
  <header>...</header>
  <main>
    <section id="list">
      <h2>Section Title</h2>
      <img ...>
      <img ... />
      ...
    </section>
  </main>
```

The click event on the <img> will also fire for each of its ascendant elements as well. Thus, we could register our click event handler in any of these ascendant elements.

Clicking on this <img> element also means you are clicking on all of its ascendant elements.

This click event thus propagates or bubbles upwards, that is, from <img> to <section> to <main> and then to <body>. We could efficiently handle the click events for the images in *any* of these ascendant elements.

Using the Dataset Property

- One of the more challenging aspects of writing JavaScript involves differences in timing between what variables are available to a function handler when it is being defined and what variables are available to that same function when it is being executed.
- The solution is to make use of the **dataset** property of the DOM element, which provides read/write access to custom data attributes (data-*) set on the element. For instance, you can make use of these via markup or via JavaScript. In markup, it can be added to any element as shown in the following:

```
<img src='file.png' id='a' data-id='5' data-country='Peru' />
```

Event Types

There are many different types of events that can be triggered in the browser. Perhaps the most obvious event is the click event, but JavaScript and the DOM support several others.

In actuality, there are several classes of event, with several types of events within each class specified by the W3C. Some of the most commonly used **event types** are:

- mouse events,
- keyboard events,
- touch events,
- form events, and
- frame events.

Mouse Events

- **click** The mouse was clicked on an element.
- **dblclick** The mouse was double clicked on an element.
- **mousedown** The mouse was pressed down over an element.
- **mouseup** The mouse was released over an element.
- **mouseover** The mouse was moved (not clicked) over an element.
- **mouseout** The mouse was moved off of an element.
- **mousemove** The mouse was moved while over an element.

Keyboard Events

Keyboard events are often overlooked by novice web developers, but are important tools for power users.

- **keydown** The user is pressing a key (this happens first).
- **keyup** The user releases a key that was down (this happens last).

```
document.getElementById("key").addEventListener("keydown", function (e) {
  // get the raw key code
  let keyPressed=e.key;
  // convert to string
  let character=String.fromCharCode(keyPressed);
  alert("Key " + character + " was pressed");
});
```

LISTING 9.7 Listener that hears and alerts key presses

Form Events

- **blur** Triggered when a form element has lost focus (i.e., control has moved to a different element), perhaps due to a click or Tab key press.
- **change** Some <input>, <textarea>, or <select> field had their value change. This could mean the user typed something, or selected a new choice.
- **focus** Complementing the blur event, this is triggered when an element gets focus (the user clicks in the field or tabs to it).
- **reset** HTML forms have the ability to be reset. This event is triggered when that happens.
- **select** When the users selects some text. This is often used to try and prevent copy/paste.
- **submit** When the form is submitted this event is triggered. We can do some prevalidation of the form in JavaScript before sending the data on to the server.

Handling the submit event

```
document.querySelector("#loginForm").addEventListener("submit",
function(e) {
        let pass = document.querySelector("#pw").value;
        if (pass=="") {
                alert ("enter a password");
                e.preventDefault();
        }
});
```

LISTING 9.8 Handling the submit event

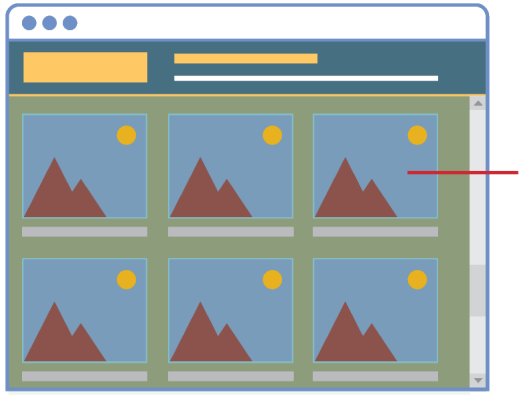
Media Events

- **ended** Triggered when playback of audio or video element is completed.
- **pause** Triggered when playback is paused.
- **play** Triggered when playback is no longer paused.
- **ratechange** Triggered when playback speed changes.
- **volumechange** Triggered when audio volume has changed.

Frame Events

- **abort** An object was stopped from loading.
- **error** An object or image did not properly load.
- **load** When window content is fully loaded.
- **DOMContentLoaded** When DOM elements in document are loaded.
- **orientationchange** The device‘s orientation has changed from portrait to landscape, or vice-versa.
- **resize** The document view was resized.
- **scroll** The document view was scrolled.
- **unload** The document has unloaded.

Lazy Loading



1 Images that are not visible will not yet be downloaded. This will improve perceived responsiveness of the visible portion of the page.

```
<img src="..."
     alt="..." >
```

```
<img data-src="..."
     data-alt="..."
     class="lazy">
```

Since no `src` attribute is provided, these images are not downloaded by the browser when HTML is received.

```
img.lazy {
   width: 320px;
   height: 240px;
    }
```

CSS for images are sized correctly so that no content shifting occurs when real images are received.

2 Event listeners will be needed for `scroll`, `resize`, and `orientationChanged` events.

```
document.addEventListener("scroll", lazyload);
window.addEventListener("resize", lazyload);
window.addEventListener("orientationChange", lazyload);
```

3 For each image, check if now visible. If it is, then change its `src` attribute to the correct one in `data-src`. This will make the browser request that file.

```
function lazyLoad() {
   ...
   const images = document.querySelectorAll("img.lazy");
   for (let img of images) {
      if (img.offsetTop < (window.innerHeight + window.pageYOffset))   {
         img.src = img.dataset.src;
         img.alt = img.dataset.alt;
         img.classList.remove('lazy');
      }
   }
```

Forms in JavaScript

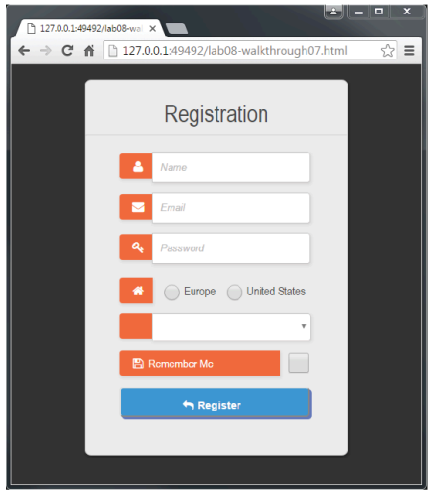
Chapter 5 covered the HTML for data entry forms.

JavaScript within forms is more than just the client-side validation of form data; JavaScript is also used to improve the user experience of the typical browser-based form.

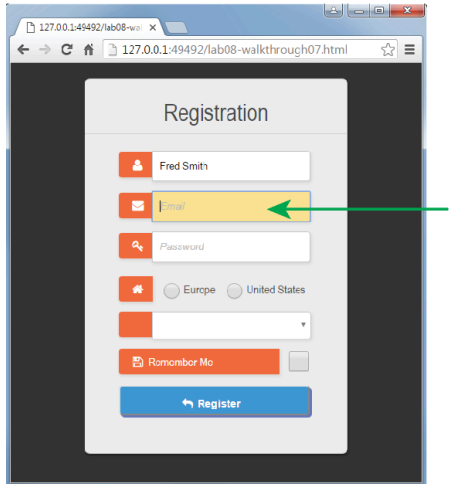
As a result, when working with forms in JavaScript, we are typically interested in three types of events:

- movement between elements,
- data being changed within a form element, and
- the final submission of the form.

Responding to form movement events



How form appears when no controls have the focus



When a control has the focus, then chang its background color

```
// This function is going to get called every time the focus or blur events are
// triggered in one of our form's input elements.
function setBackground(e) {
   if (e.type == "focus") {
      e.target.style.backgroundColor = "#FFE393";
   }
   else if (e.type == "blur") {
      e.target.style.backgroundColor = "white";
   }
}

// set up the event listeners only after the DOM is loaded
window.addEventListener("load", function() {
   const cssSelector = "input[type=text],input[type=password]";
   const fields = document.querySelectorAll(cssSelector);
   for (let f of fields) {
      f.addEventListener("focus", setBackground);
      f.addEventListener("blur", setBackground);
   }
});
```

Here we use the `style` property instead of the `classList` property because of specificity conflicts (that is, attribute selectors override class selectors).

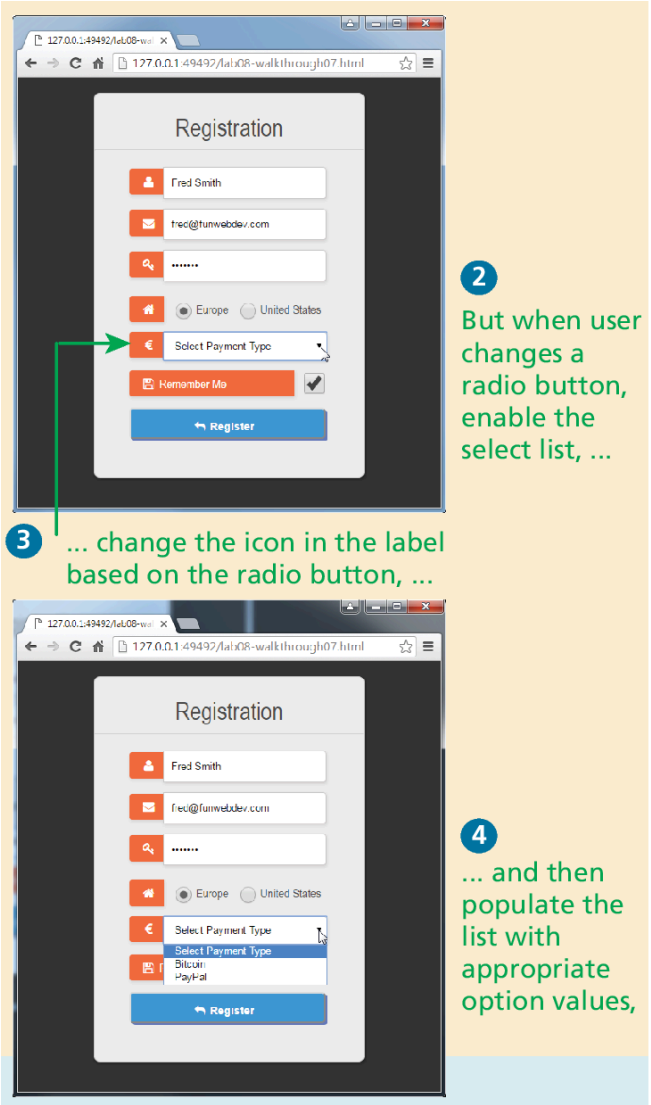
Selects the fields that will change.

Assigns the `setBackground()` function to change the background color of the control depending upon whether it has the focus.

Responding to Form Changes Events

We may want to change the options available within a form based on earlier user entry. For instance, we may want the payment options to be different based on the value of the region radio button.

Figure 9.19 demonstrates how we can add event listeners to the change event of the radio buttons; when one of these buttons changes its value, then the callback function will set the available payment options based on the selected region. The listing also changes the associated payment label as well.



Validating a Submitted Form

Form validation continues to be one of the most common applications of JavaScript.

Checking user inputs to ensure that they follow expected rules must happen on the server side for security reasons (in case JavaScript was circumvented); checking those same inputs on the client side using JavaScript will reduce server load and increase the perceived speed and responsiveness of the form.

Some of the more common validation activities include email validation, number validation, and data validation.

In practice, regular expressions (covered in Section 9.6) are used to concisely implement many of these validation checks.

Empty Field Validation

```
const form = document.querySelector("#loginForm");
form.addEventListener("submit", (e) => {
   const fieldValue = document.querySelector("#username").value;
   if (fieldValue == null || fieldValue == "") {
      // the field was empty. Stop form submission
      e.preventDefault();
      // Now tell the user something went wrong
      console.log("you must enter a username");
   }
});
```

LISTING 9.10 A simple validation script to check for empty fields

Determining which items in multiselect list are selected

```
const multi = document.querySelector("#listbox");
// using the options technique loops through each option and check if it is selected
for (let i=0; i < multi.options.length; i++) {
    if (multi.options[i].selected) {
        // this option was selected, do something with it ...
        console.log(multi.options[i].textContent);
    }
}

// the selectedOptions technique is simpler ... it only loops through the selected options
for (let i=0; i < multi.selectedOptions.length; i++) {
    console.log(multi.selectedOptions[i].textContent);
}
```

LISTING 9.11 Determining which items in multiselect list are selected

Number Validation

Unfortunately, no simple functions exist for number validation like one might expect from a fullfledged library. Using parseInt(), isNAN(), and isFinite(), you can write your own number validation function.

Validating email, phone numbers, or social security numbers would include checking for blank fields and making use of isNumeric and regular expressions.

```
function isNumeric(n) {
        return !isNaN(parseFloat(n)) && isFinite(n);
}
```

LISTING 9.12 A function to test for a numeric value

Submitting Forms

Submitting a form using JavaScript requires having a node variable for the form element. Once the variable, say, *formExample* is acquired, one can simply call the **submit()** method:

```
const formExample = document.getElementById("loginForm");
formExample.submit();
```

This is often done in conjunction with calling preventDefault() on the submit event.

It is possible to submit a form multiple times by clicking buttons quickly. The easiest way to protect against this is to simply disable the submit button immediately in the event handler for the submit event.

Regular Expressions

A **regular expression** is a set of special characters that define a pattern.

Their history predates the world of web development, as evidenced by the formal specification defined by the IEEE POSIX standard.

PHP, JavaScript, Java, the .NET environment, and most other modern languages support regular expressions.

Regular Expression Syntax

A regular expression consists of two types of characters: literals and metacharacters.

A **literal** is just a character you wish to match in the target (i.e., the text that you are searching within).

A **metacharacter** is a special symbol that acts as a command to the regular expression parser (there are 14, listed below).

. [ ] \ ( ) ^ $ | * ? { } +

Table 9.12 lists examples of typical metacharacter usage to create patterns;

Regular Expression Syntax (ii)

In JavaScript, regular expressions are case sensitive and contained within forward slashes. For instance

let pattern = /ran/;

will find matches in all three of the following strings:

'randy connolly'

'Sue ran to the store'

'I would like a cranberry'

Listing 9.13 contains a more complex regular expression whose development is described in textbook

Key Terms

blur

Document Object Model (DOM)

document root

DOM document

object

DOM tree

Element Node

event bubbling

phase

event capturing phase

event delegation

event handler

event object

event propagation

event target

event type

focus

form events

frame events

iteral

keyboard events

linter

media events

metacharacter

mouse events

node

nodeList

regular expression

selection methods

Fundamentals of Web Development

Third Edition by Randy Connolly and Ricardo Hoar

Chapter 10

JavaScript 3:

Additional Features

In this chapter you will learn . . .

- Additional language features in JavaScript
- How to asynchronously consume web APIs in JavaScript
- Extend the capabilities of your pages using browser APIs
- Utilize external APIs for mapping and charting

Array Functions

- **forEach()** iterate through an array
- **find()** find the *first* object whose property matches some condition
- **filter()** find all matches whose property matches some condition
- **map()** is similar manner to filter except it creates a new array of the same size whose values have been transformed by the passed function
- **reduce()** reduces an array into a single value
- **sort()** sorts a one-dimensional array

Array forEach()

This function will be called for each element in the array

```
paintings.forEach( (p) => {
  console.log(p.title + ' by ' + p.artist)
} );
```

Each element is passed in as an argument to the function.

```
const paintings = [
  {title: "Girl with a Pearl Earring", artist: "Vermeer"},
  {title: "Artist Holding a Thistle", artist: "Durer"},
  {title: "Wheatfield with Crows", artist: "Van Gogh"},
  {title: "Burial at Ornans", artist: "Courbet"},
  {title: "Sunflowers", artist: "Van Gogh"}
];
```

Array find()

One of the more common coding scenarios with an array of objects is to find the *first* object whose property matches some condition. This can be achieved via the **find**() method of the array object, as shown below.

```
const courbet = paintings.find( p => p.artist === 'Courbet' );
console.log(courbet.title); // Burial at Ornans
```

Like the **forEach**() method, the **find**() method is passed a function; this function must return either true (if condition matches) or false (if condition does not match). In the example code above, it returns the results of the conditional check on the artist name.

Array filter()

If you were interested in finding all matches you can use the filter() method, as shown in the following:

```
// vangoghs will be an array containing two painting objects
const vangoghs = paintings.filter(p => p.artist === 'Van Gogh');
```

Since the function passed to the filter simply needs to return a true/false value, you can make use of other functions that return true/false. For instance, you could perform a more sophisticated search using regular expressions.

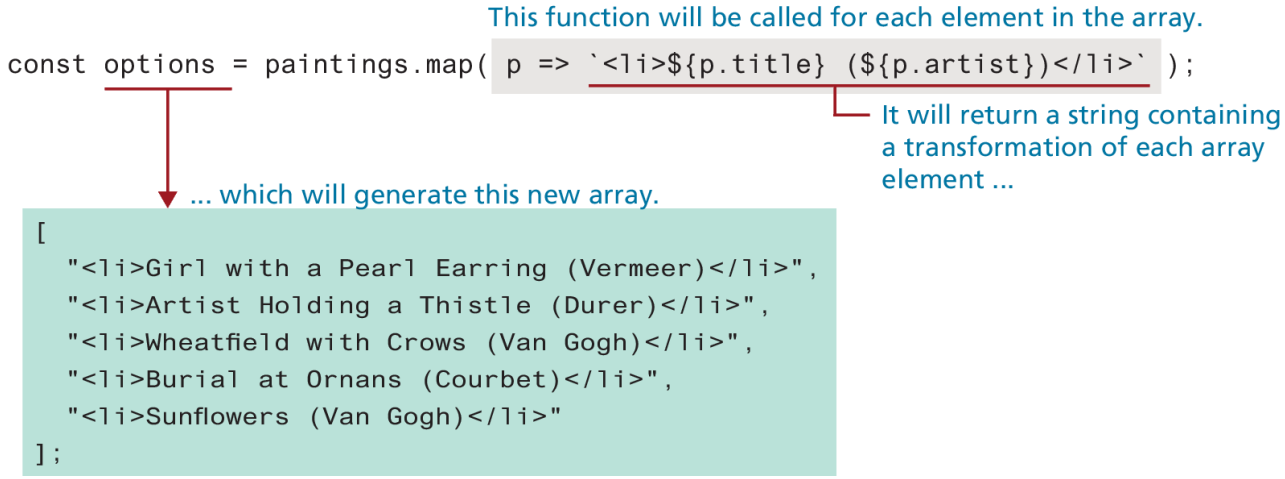
Array map()

The map() function operates in a similar manner except it creates a new array of the same size but whose values have been transformed by the passed function.

Listing 10.2 shows map using DOM nodes. Figure 10.2 uses strings.

```
// create array of DOM nodes
const options = paintings.map( p => {
    let item = document.createElement("li");
    item.textContent = `${p.title} (${p.artist})`;
    return item;
});
```

LISTING 10.2 Using the map() function



Reduce

The **reduce**() function is used to reduce an array into a single value. Like the other array functions in this section, the **reduce**() function is passed a function that is invoked for each element in the array.

This callback function takes up to four parameters, two of which are required: the previous accumulated value and the current element in the array.

For instance, the following example illustrates how this function can be used to sum the **value** property of each painting object in our sample paintings array:

```
let initial = 0;

const total = paintings.reduce( (prev, p) => prev + p.value, initial);
```

Sort

sort() function sorts in ascending order (after converting to strings if necessary)

```
const names = ['Bob', 'Sue', 'Ann', 'Tom', 'Jill'];
const sortedNames = names.sort();
// sortedNames contains ["Ann", "Bob", "Jill", "Sue", "Tom"]
```

If you need to sort an array of objects based on one of the object properties, you will need to supply the sort() method with a compare function that returns either 0, 1, or −1, depending on whether two values are equal (0), the first value is greater than the second (1), or the first value is less than the second (−1).

Custom sort

```
const sortedPaintingsByYear = paintings.sort( function(a,b) {
   if (a.year < b.year)
      return -1;
   else if (a.year > b.year)
      return 1;
   else
      return 0;
} );

// more concise version using ternary operator and arrow syntax
const sorted2 = paintings.sort( (a,b) => a.year < b.year? -1: 1 );
```

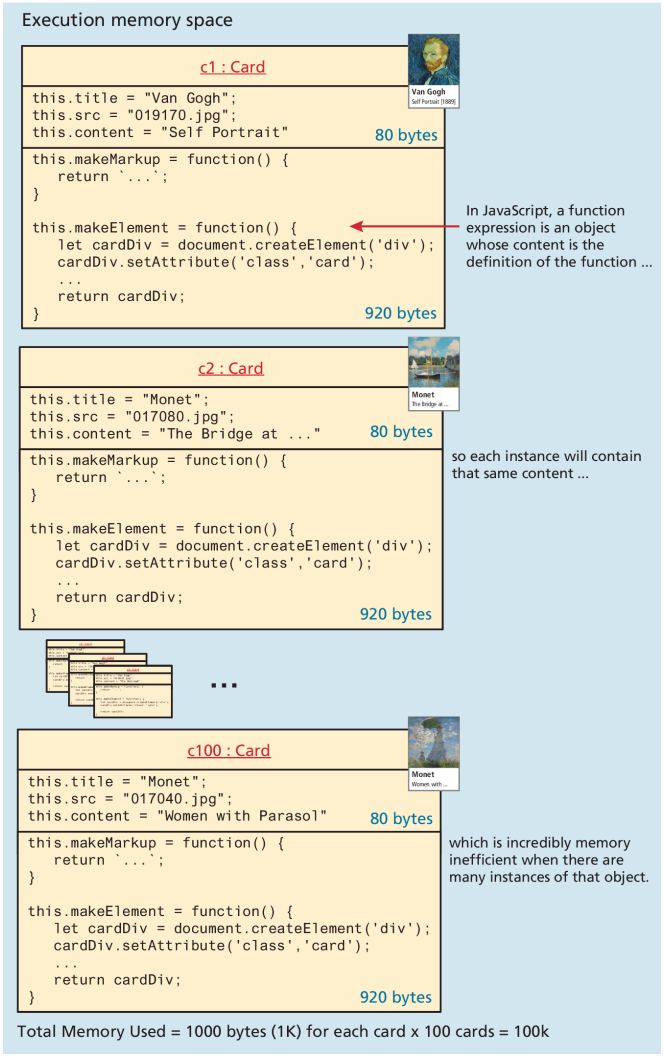
LISTING 10.3 Sorting an array based on the properties of an object

Prototypes, Classes, and Modules

In Chapter 8, you learned how to use constructor functions as an approach for creating multiple instances of objects that need to have the same properties.

While the constructor function is simple to use, it can be an inefficient approach for objects that contain methods since memory must be allocated for each (identical) method.

Prototypes help address this issue.



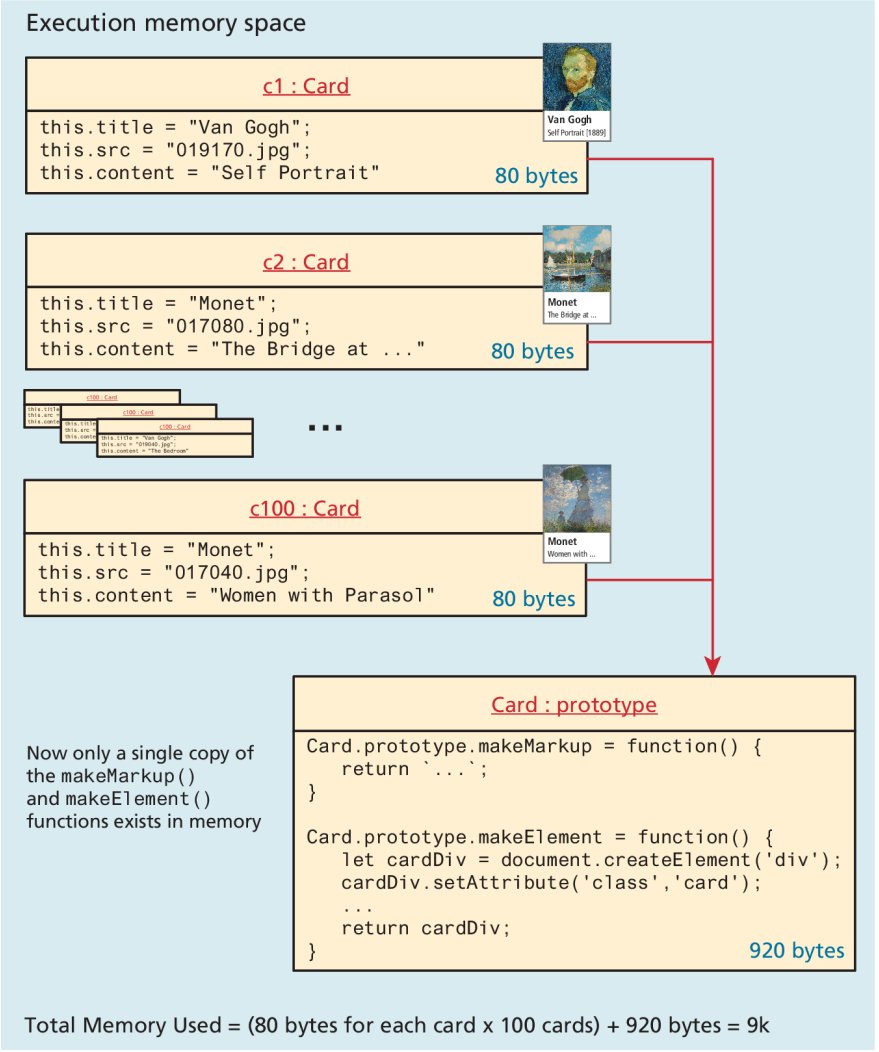
Prototypes

Prototypes make JavaScript behave more like an object-oriented language.

Every function object has a **prototype** property, which is initially an empty object.

The prototype properties are defined once for all instances of an object created with the new keyword from a constructor function.

This approach is far superior because it defines the method only once, no matter how many instances are created.



Using a prototype

```
function Card(title, src, content) {
    this.title = title;
    this.src = src;
    this.content = content;
}

Card.prototype.makeMarkup = function() {
    return `<div class="card">
            <img src="${this.src}" alt="${this.title}" >
            <div>
                <h4>${this.title}</h4>
                <p>${this.content}</p>
            </div>
            </div>`;
};
```

```
Card.prototype.makeElement = function() {
    let cardDiv = document.createElement('div');
     ....
    cardDiv.appendChild(div);
    return cardDiv;
};

// You use prototype functions as if they were declared
in the object
const container =
document.querySelector("#container");
const c1 = new Card("Van Gogh", "019170.jpg", "Self
Portrait");
container.appendChild( c1.makeElement() );
```

LISTING 10.5 Using a prototype

Using Prototypes to Extend Other Objects

Prototypes also enable you to extend existing objects (including built-in objects) by adding to their prototypes. Imagine a method added to the String object that allows you to count instances of a character.

```
String.prototype.countChars = function (c) {
                let count=0;
                for (let i=0;i<this.length;i++) {
                                if (this.charAt(i) == c)
                                                count++;
                }
                return count;
}
```

```
const msg = "HELLO WORLD";
console.log(msg + " has" +
msg.countChars("L") + " letter L's");
```

LISTING 10.6 Extending a built-in object using the prototype

Classes

- A **class** provides an alternate syntax for a function constructor and the extension of it via its prototype. In reality, they are merely “syntactical sugar” for JavaScript’s prototype approach to inheritance
- While the class syntax provides a familiar alternate syntax for working with functions, the developer community has not universally adopted it
- Regardless of these concerns, the React framework, which has become one of the most widely adopted frameworks in the past several years (and which is covered in Chapter 11), does use JavaScript class syntax, so it is likely that as a JavaScript developer you will encounter this syntax more and more moving forward.

Using a class

```
class Card {
   // constructor replaces the function constructor
   constructor(title, src, content) {
   this.title = title;
   this.src = src;
   this.content = content;
   }
   // class methods replace prototypes
   makeMarkup() {
   return `<div class="card">
         <img src="${this.src}" alt="${this.title}" >
         <div>
            <h4>${this.title}</h4>
            <p>${this.content}</p>
         </div>
         </div>`;
   };
   // notice the function property shorthand syntax
   makeElement() {
      let cardDiv = document.createElement('div');
      …
      return cardDiv;
   }
}

// Use the class
const container =
document.querySelector("#container");
const c1 = new Card("Van Gogh", "images/019170.jpg",
"Self Portrait");
container.append( c1.makeElement() );
```

LISTING 10.7 Implementing Listing 10.5 (slide 13) using class syntax

Extending classes and more

There are additional syntactical features of classes in JavaScript, including getters/ setters and static functions that we are not covering.

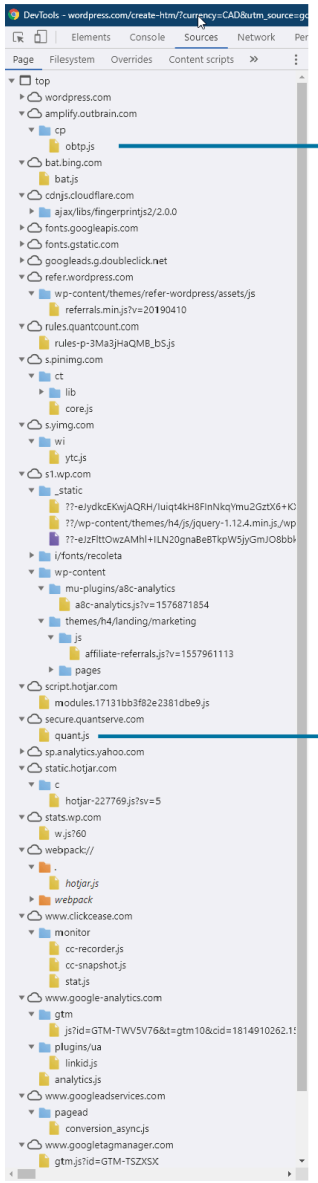
Extending classes is one advanced feature worth noting. The **extends** keyword lets a class inherit the properties and methods of another class as with class-based programming languages such as Java or C#

```
class AnimatedCard extends Card {
  constructor(title, src, content, effect) {
  super(title, src, content)
  this.effect=effect;
}
```

Name Conflicts

As shown in Figure 10.5, complex contemporary JavaScript applications might contain hundreds of literals defined in dozens of .js files, so some way of preventing name conflicts becomes especially important

wordpress.com Home Page (Jan 2020)



This is one of 23 external JavaScript libraries used by this page.

```
// imagine if this library contained this ...
function calculate(x,y,z) {
    ...
}
let result = calculate(foo,bar,can);
```

This code would then call the most recently defined version of this function and not the one within its own library, almost certainly resulting in some type of error or bug.

```
// and this library contained this ...
function calculate() {
    ...
}
```

Name conflict!

This version would replace any previously-defined `calculate()` functions.

Modules

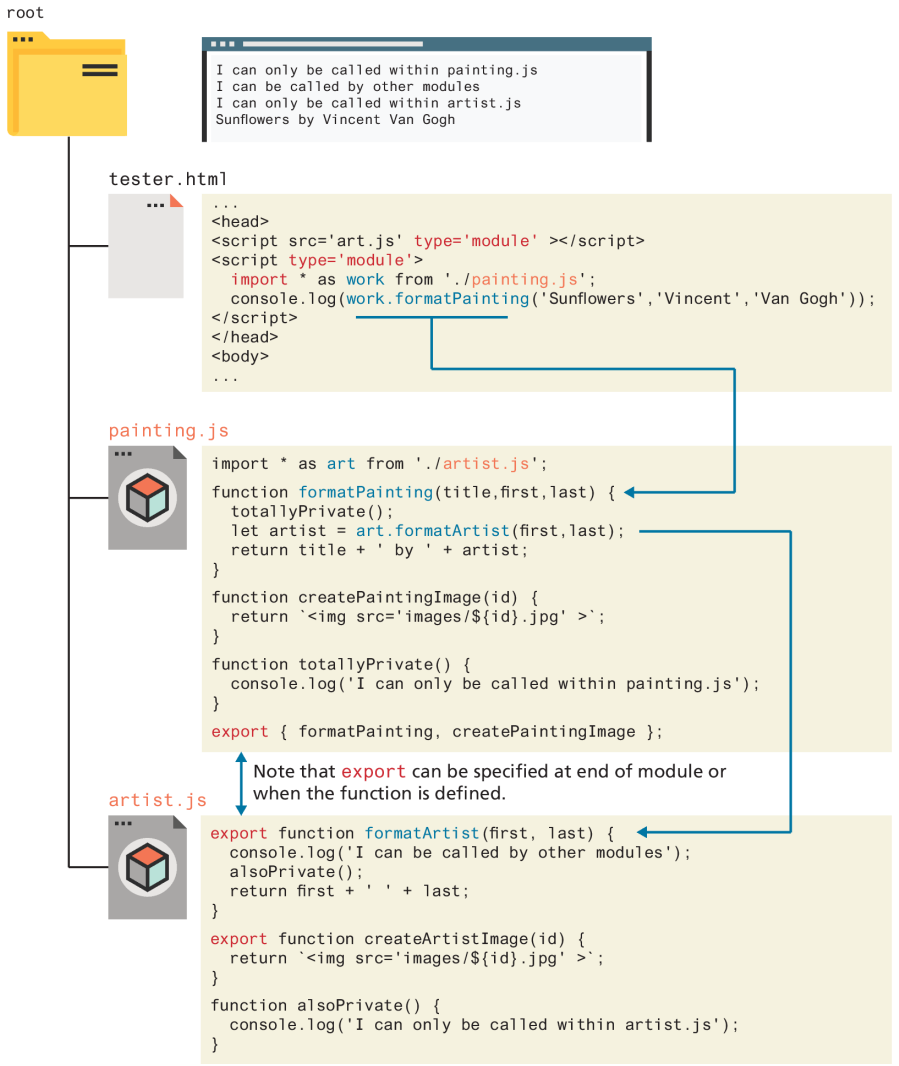
An ES6 **module** is simply a file that contains JavaScript. Unlike a regular JavaScript external file, literals defined within the module are scoped to that module.

You do have to tell the browser that a JavaScript file is a module and not just a regular external JavaScript file within the <script> element. This is achieved via the type attribute as shown in the following:

```
<script src="art.js" type="module"></script>
```

Within a module, any literals are private to that module. To make content in the module file available to other scripts outside the module, you have to make use of the **export** keyword.

Visualizing Modules in JavaScript



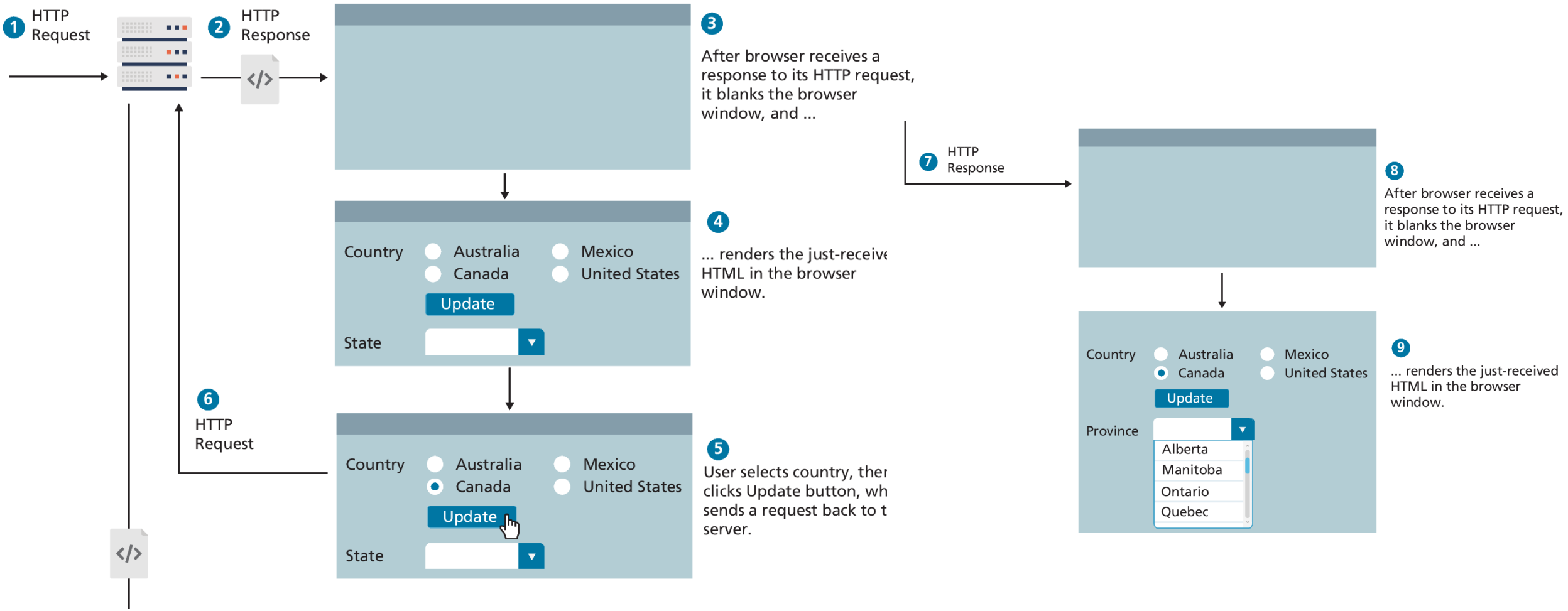
Asynchronous Coding with JavaScript

asynchronous code is code that is doing (or seemingly doing) multiple things at once. In multi-tasking operating systems, asynchronous execution is often achieved via **threads**: each thread can do only one task at a time, but the operating system switches between threads

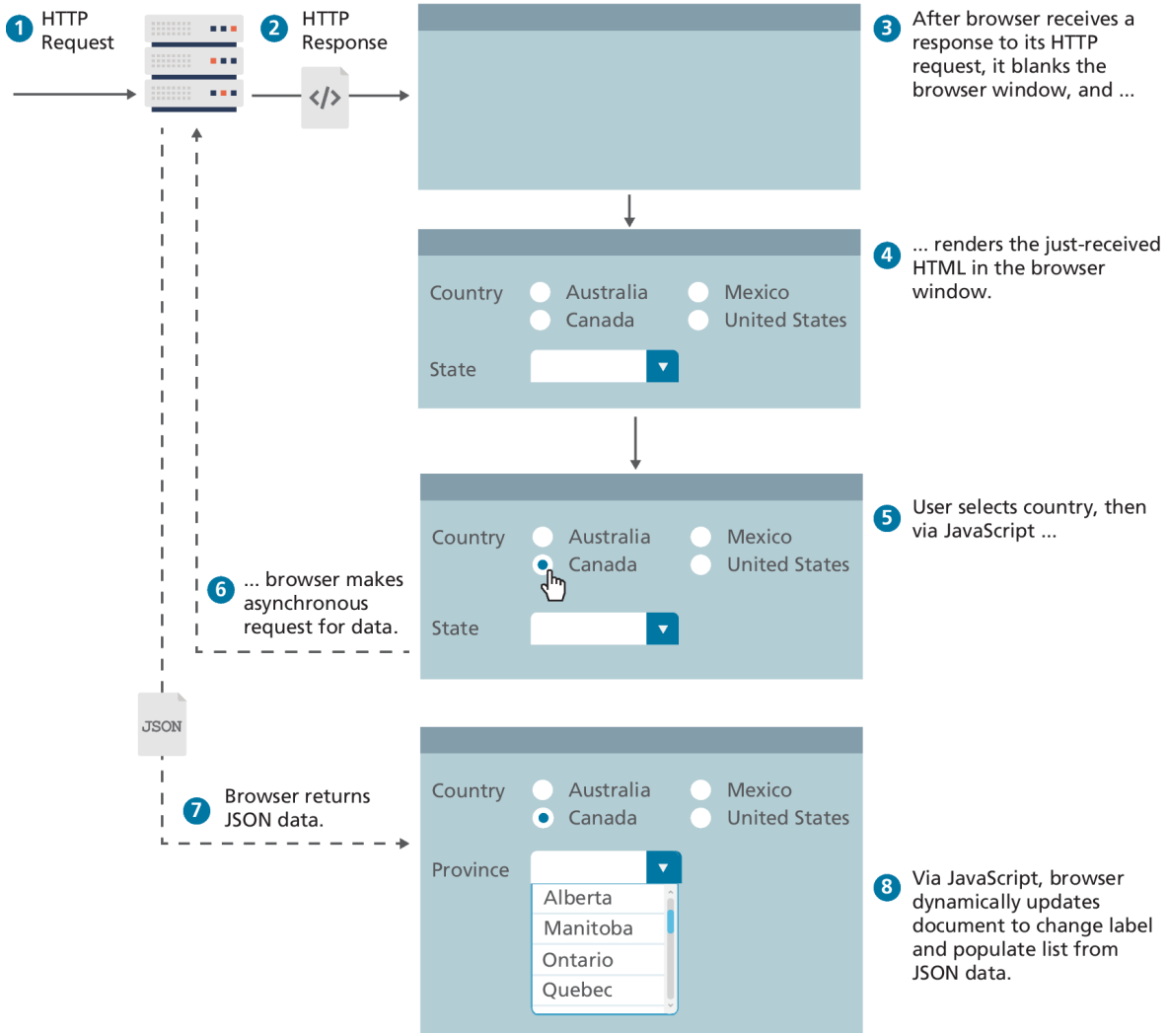
Many contemporary web sites make use of asynchronous JavaScript data requests of Web APIs, thereby allowing a page to be dynamically updated without requiring additional HTTP requests.

A **web API** is simply a web resource that returns data instead of HTML, CSS, JavaScript, or images

Normal HTTP request–response loop



Asynchronous data requests

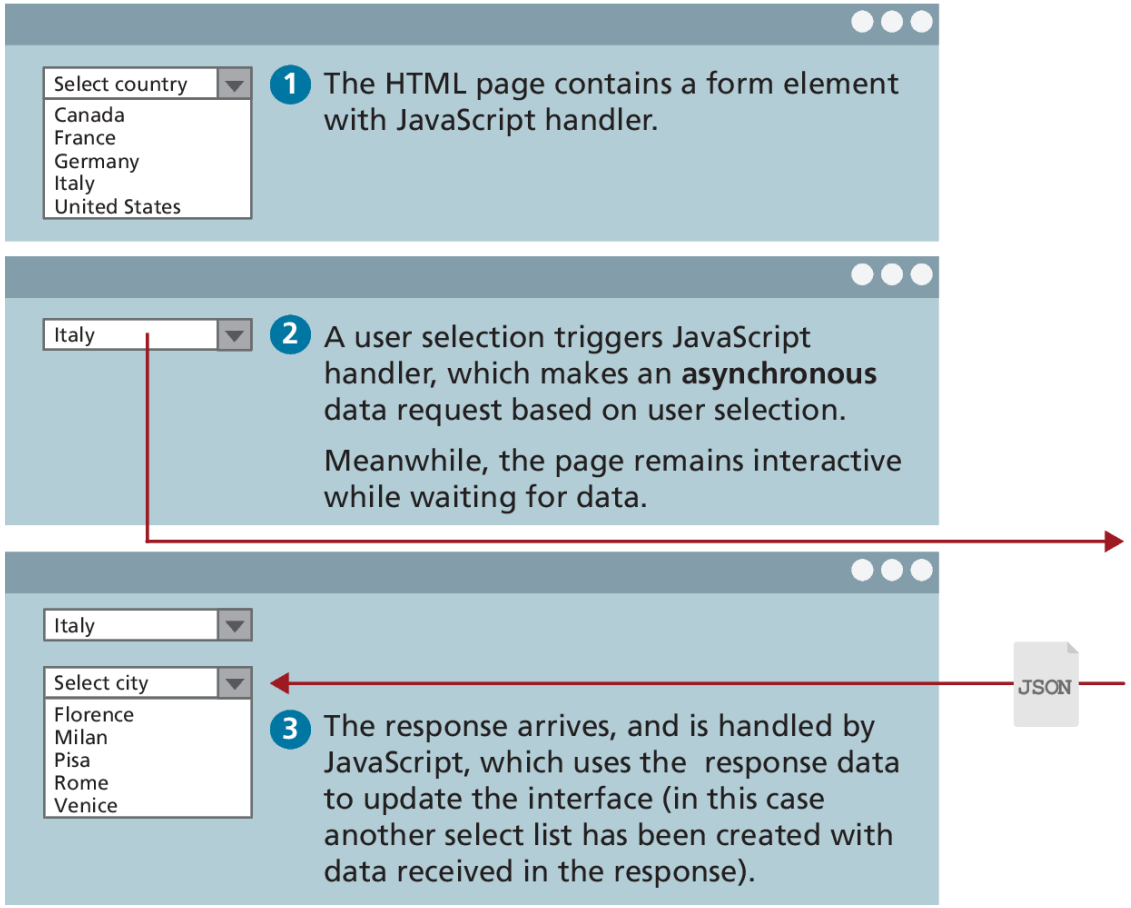


Fetching Data from a Web API

To illustrate fetch, consider the scenario of a page containing a <select> element.

When the user selects from the country list, the page makes an asynchronous request to retrieve a list of cities for that country.

```
let cities = fetch('/api/cities.php?country=italy');
```



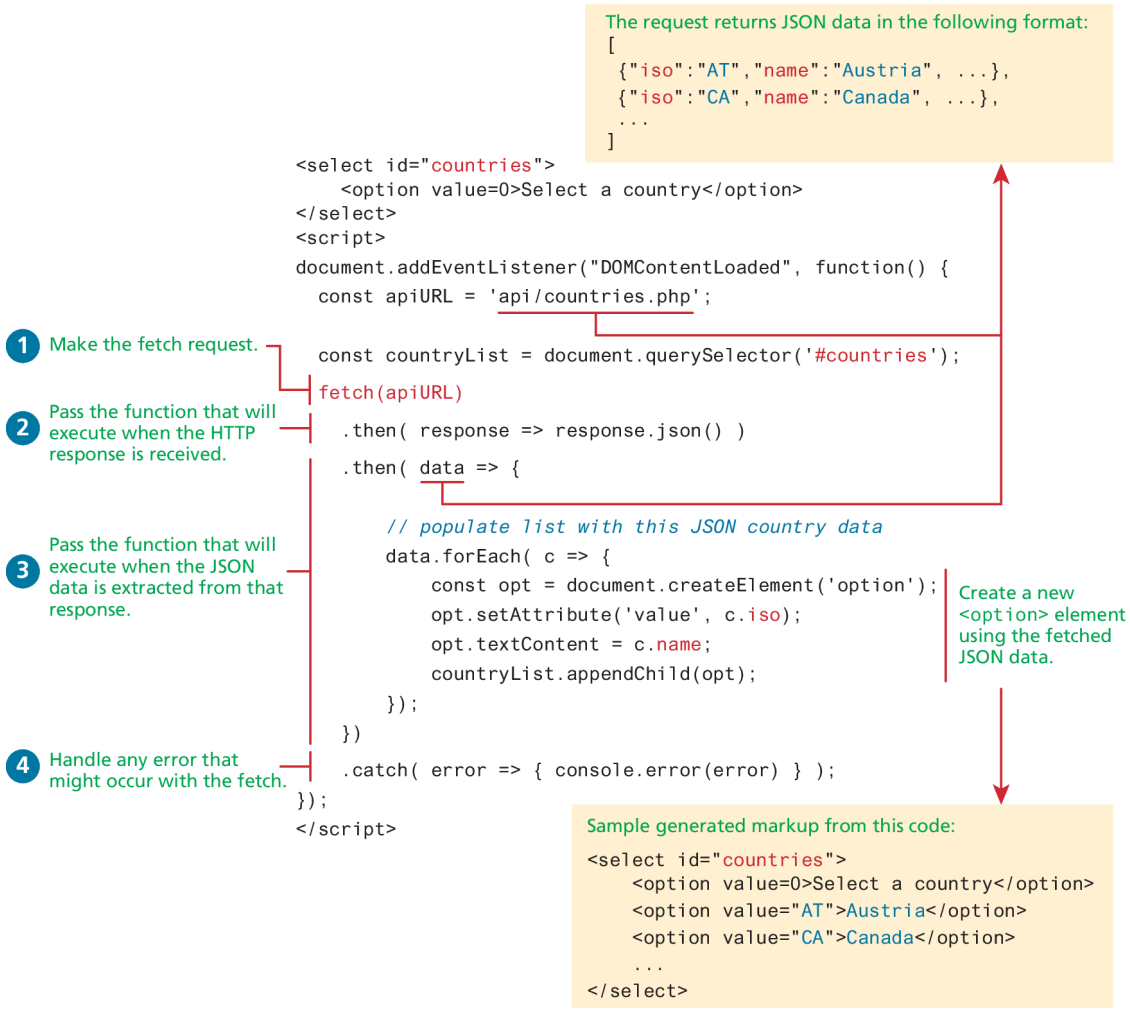
Fetching Data from a Web API (ii)

So what does cities contain after this call? You might expect it to contain the requested JSON data. But it doesn't. Why? Because it will take time for this service to execute and respond to our request.

What the above fetch will return instead is a special Promise object.

Promises in JavaScript are usually handled by invoking two methods: then() for responding to the successful arrival of data, and catch() for responding to an unsuccessful arrival.

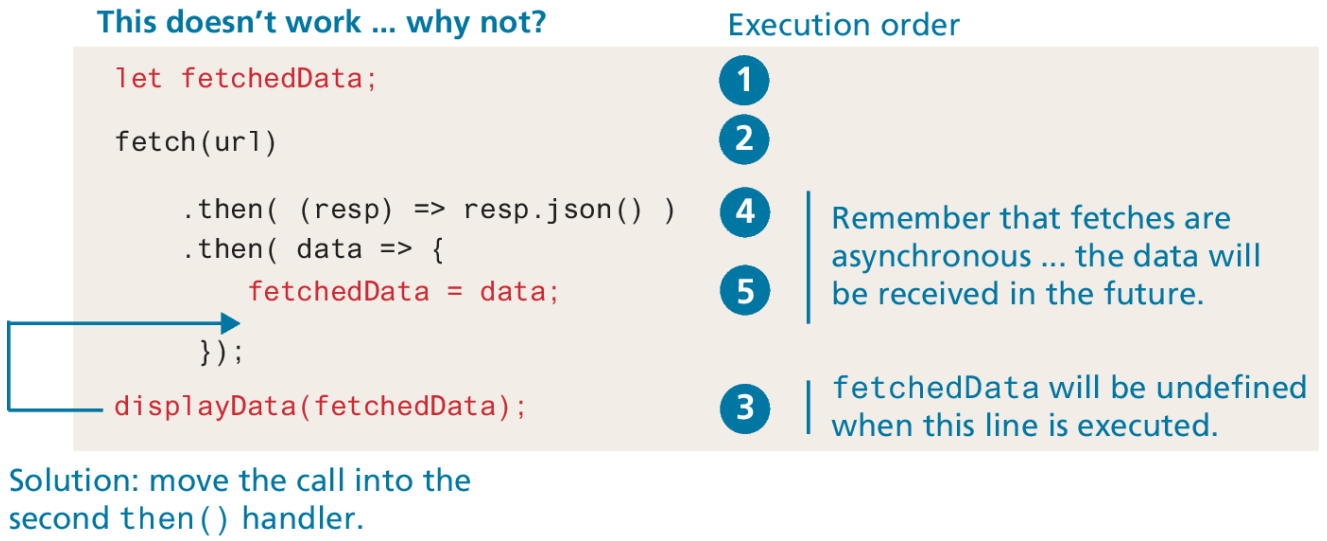
Example of asynchronous request using fetch



Common Mistakes with Fetch

Students often struggle at first with using fetch and often commit some version of the mistake shown in Figure 10.14.

Multiple nested fetches can be problematic,



Cross-Origin Resource Sharing

Modern browsers prevent cross-origin requests by default (Chapter 16), so sharing content legitimately between two domains becomes harder.

Cross-origin resource sharing (CORS) is a mechanism that uses new HTTP headers in the HTML5 standard that allows a JavaScript application in one **origin** (i.e., a protocol, domain, and port) to access resources from a different origin.

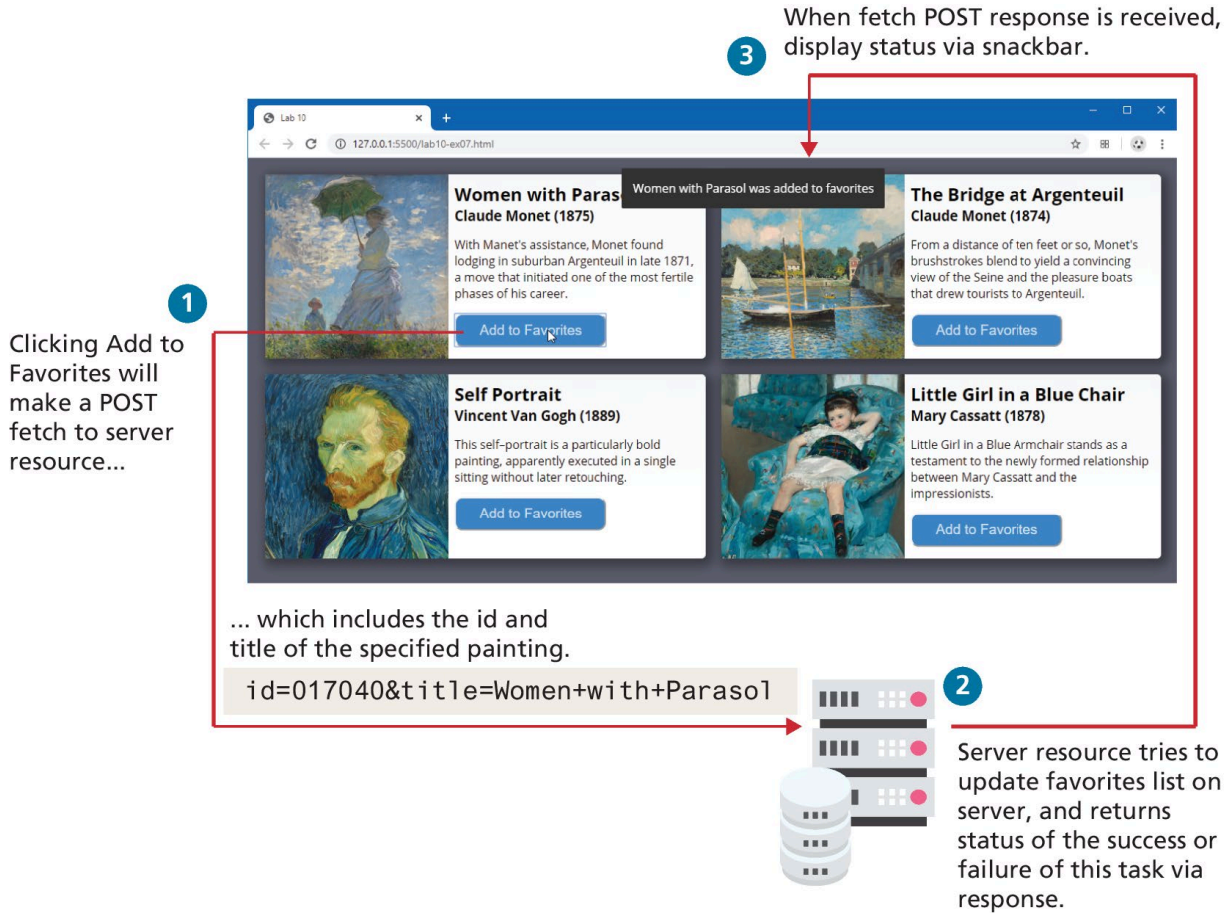
If an API site wants to allow any domain to access its content through JavaScript, it would add the following header to all of its responses:

Access-Control-Allow-Origin: *

Fetching Using Other HTTP Methods

By default, fetch uses the HTTP GET method. There are times when you will instead want to use POST, or even PUT or DELETE

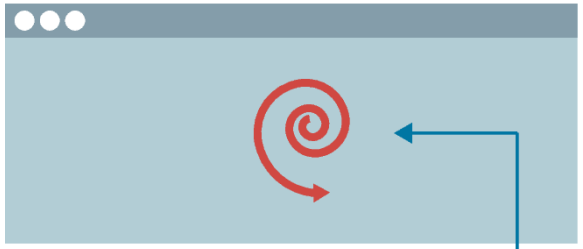
For instance, imagine you wanted to add an item to a favorites list or to a shopping cart in an asynchronous manner. This would typically require sending data to the server, so a POST fetch makes the most sense.



Adding a Loading Animation

Fetching takes time. A common user interface feature is to supply the user with a loading animation while the data is being fetched.

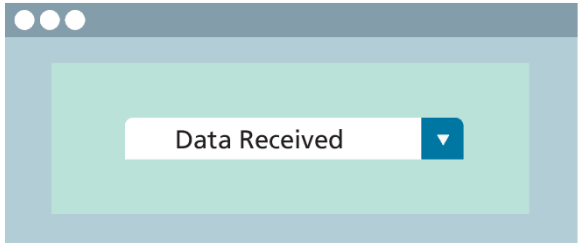
Simply show or hide an element that contains either an animated GIF, or, even better, CSS that uses animation



```
<div id="load"></div>
<div id="box">
   ...
</div>
```

A CSS animation is defined for this id

This div will eventually display fetched data



1 Before fetch hide content box and show loading div.

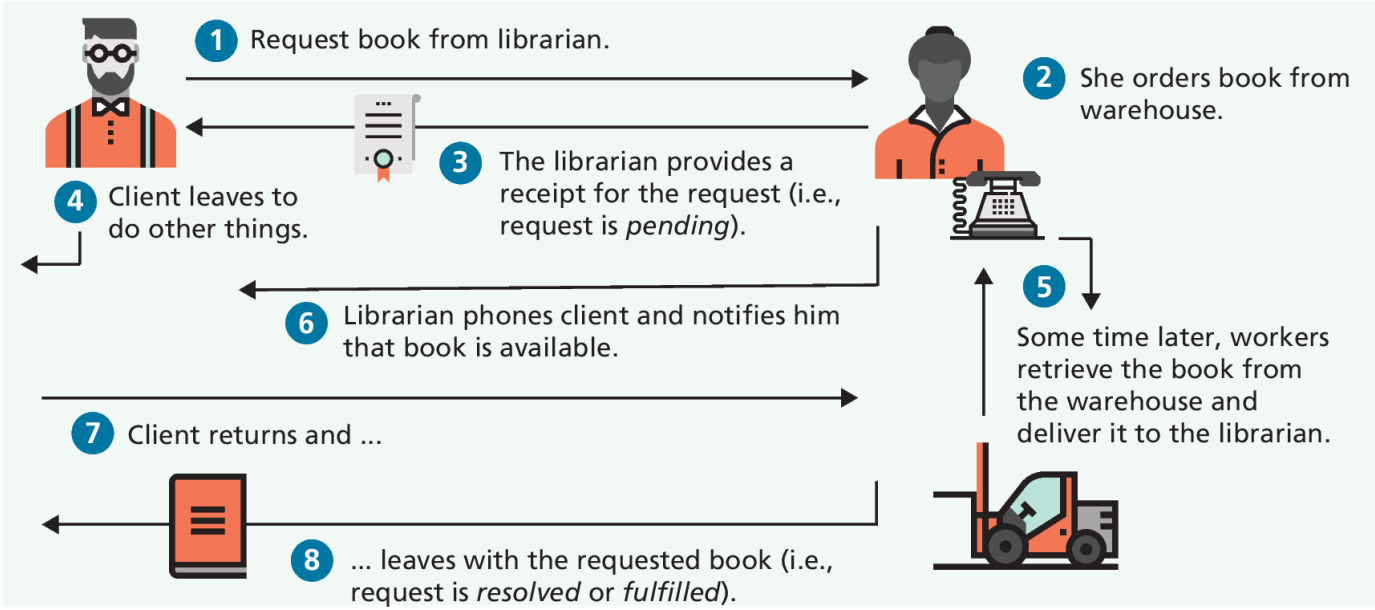
```
let box = document.querySelector("#box");
let load = document.querySelector("#load");
// hide box and display loading animation
box.style.display = "none";
load.style.display = "block";
```

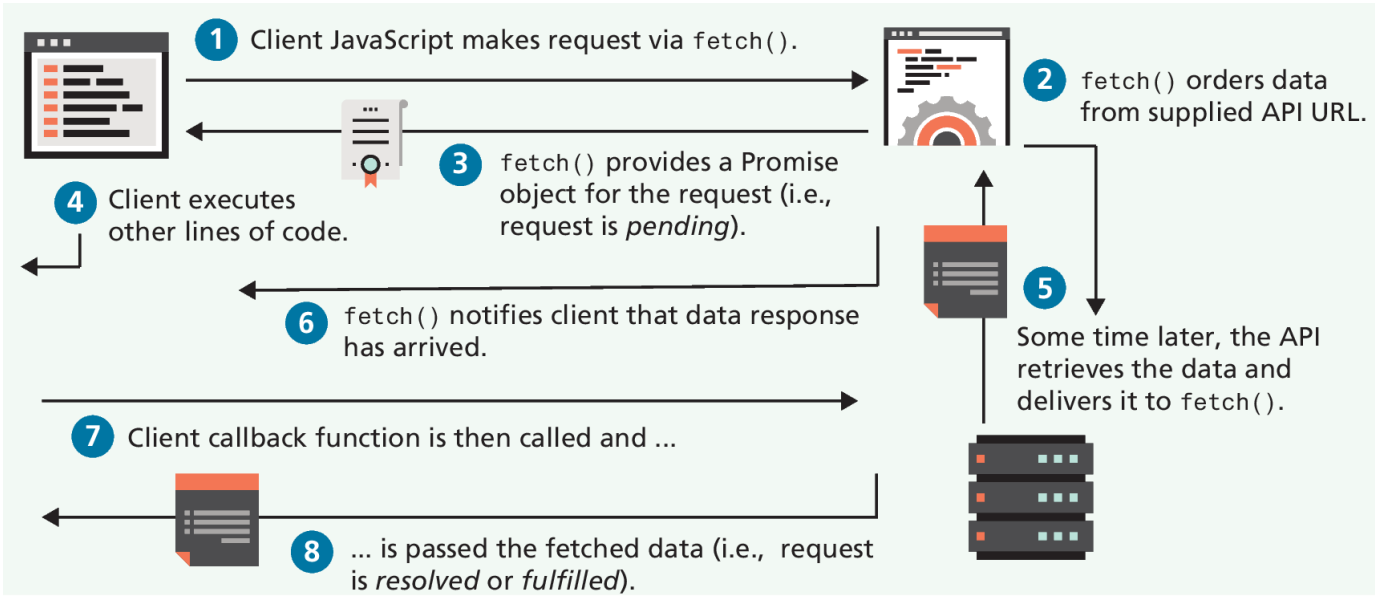
2 Once fetch data is received, hide animation and show box.

```
fetch(endpoint)
  .then(response => response.json())
  .then(data => {
     load.style.display = "none";
     box.style.display = "block";
     // do other stuff with data
     ...
});
```

Promises

A **Promise** is a placeholder for a value that we don't have now but will arrive later. Eventually, that promise will be completed and we will receive the data, or it won't, and we will get an error instea,





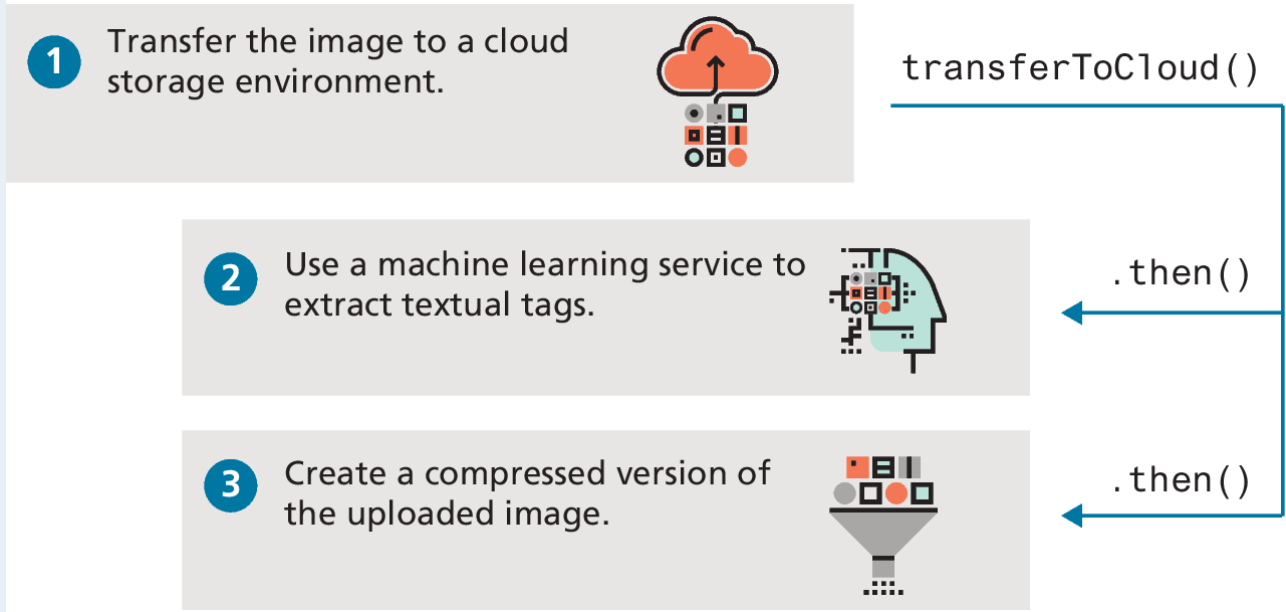
Creating a Promise

Creating a promise is quite simple: you simply instantiate a Promise object.
For promises to make some sense, we must first understand that the handler function passed to the Promise constructor must take two parameters: a **resolve**() function and a **reject**() function.

```
const promiseObj = new Promise( (resolve, reject) => {
 if (someCondition)
    resolve(someValue);
 else
    reject(someMessage);
 });
promiseObj
    .then( someValue => {
        // success, promise was achieved!
    })
    .catch( someMessage => {
        // oh no, promise was not satisfied!!
    });
```

A Promise example

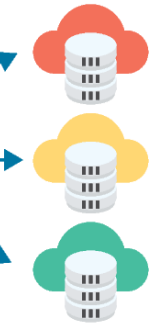
```
// promisified version of the transfer task
function transferToCloud(filename) {
    return new Promise( (resolve, reject) => {
        // just have a made-up AWS url for now
        let cloudURL ="http://.../makebelieve.jpg";
        // if passed filename exists then upload ...
        if ( existsOnServer(filename) ) {
            performTransfer(filename, cloudURL);
            resolve(cloudURL);
        } else {
            reject( new Error('filename does not exist'));
        }
    });
}
// use this function
transferToCloud(file)
.then( url => extractTags(url) )
.then( url => compressImage(url) )
.catch( err => logThisError(err) );
```



LISTING 10.10 Creating Promises

Executing multiple Promises in parallel

For executing multiple promises, you can make use of the **Promise.all()** method, which returns a single Promise when a group of Promise objects have been resolved.



```
function getData() {
   let prom1 = fetch(movieAPI).then( response => response.json() );
   let prom2 = fetch(artAPI).then( response => response.json() );
   let prom3 = fetch(langAPI).then( response => response.json() );
   return Promise.all([prom1, prom2, prom3]);
}
```

returns a Promise

passed an array of Promise objects

When *all* the passed Promise objects are resolved, then this function will be called and passed an array of the resolved data.

```
getData().then( arrayofResolves => {
  [movies, galleries, languages] = arrayofResolves;
  result.innerHTML =
    `<p>This data is from three separate fetches ...</p>
     <ul>
       <li>${movies[0].title}</li>
       <li>${galleries[0].galleryName}</li>
       <li>${languages[0].name}</li>
     </ul>`;
});
```

Uses array destructuring, to create three variables containing the data from each fetched API

Async and Await

ES7 introduced the **async...await** keywords that can both simplify the coding and even eliminate the typical nesting structure of typical asynchronous coding.

Recall this sample line from the earlier section on fetch?

```
let obj = fetch(url);
```

Content is contained in the variable obj is a **Promise**; the then() method of the Promise object needs to be called and passed a callback function that will use the data from the *promisified* function fetch.

Async and Await (ii)

The **await** keyword provides exactly that functionality, namely, the ability to treat asynchronous functions that return Promise objects as if they were synchronous.

```
let obj = await fetch(url);
```

Now, obj will contain whatever the resolve() function of the fetch() returns, which in this case is the response from the fetch. Notice that no callback function is necessary!

There is an important limitation with using the **await** keyword: it *must* occur within a function prefaced with the **async** keyword

Using Browser APIs

In the last section, you learned how to use the fetch() method to access data from external APIs. In this section, you will instead make use of the **browser APIs**

In recent years, the amount of programmatic control available to the JavaScript developer has grown tremendously. You can now, for instance, retrieve location information, access synthesized voices, recognize and transcribe speech, and persist data content in the browser's own local storage. Table 10.1 lists several of the more important browser APIs.

Web Storage API

The **Web Storage API** provides a mechanism for preserving non-essential state across requests and even across sessions. It comes in two varieties:

- **localStorage** is a dictionary of strings that lasts until removed from the browser.
- sessionStorage is also a dictionary of strings but only lasts as long as the browsing session.

To add a string to either involves calling the **setItem**() method of the **localStorage** or **sessionStorage** objects.

To retrieve a value from either simply requires using the **getItem**() method.

Web Storage API (ii)

The **Web Storage API** provides a mechanism for preserving non-essential state across requests and even across sessions. It comes in two varieties:

- **localStorage** is a dictionary of strings that lasts until removed from the browser.
- sessionStorage is also a dictionary of strings but only lasts as long as the browsing session.

To add a string to either involves calling the **setItem**() method of the **localStorage** or **sessionStorage** objects.

To retrieve a value from either simply requires using the **getItem**() method.

Web Speech API

The Web Speech API provides a mechanism for turning text into speech (sounds) and for turning speech (microphone input) into text.

To verbalize a string of text, you can simply make use of the **SpeechSynthesisUtterance** and **speechSynthesis** objects:

```
const utterance = new SpeechSynthesisUtterance('Hello world');
speechSynthesis.speak(utterance);
```

Some browsers provide different voices: for instance, U.S. male, U.S. female, U.K. male, etc. You can also adjust the speed and pitch of the speech.

GeoLocation

The **Geolocation API** provides a way for JavaScript to obtain the user's location (accuracy/availability dependent on permission and device)

```
if (navigator.geolocation) {
   navigator.geolocation.getCurrentPosition(hav
   eLocation,geoError);
} else {
// geolocation not supported or accepted
...
}


function haveLocation(position) {
   const latitude = position.coords.latitude;
   const longitude = position.coords.longitude;
   const altitude = position.coords.altitude;
   const accuracy = position.coords.accuracy;
   // now do something with this information
   ...
}
function geoError(error) { ... }
```

LISTING 10.13 Sample GeoLocation API usage

Using External APIs

An **external API** refers to objects with events and properties that perform a specific task that you can use in your pages.

Unlike browser APIs, these external APIs are **not** built into the browser but are external JavaScript libraries that need to be downloaded or referenced and added to a page via a <script> tag.

In this section, we will look at two of the most popular ones: the Google Maps API and the plotly API.

Google Maps

The Google Maps code used in the 1st edition of this book (2014) no longer worked by the time of the second edition (2017). That code no longer works now at the time of writing (2020). Hopefully, when you use this edition, the Google Maps code still works—but it might not!

- The point here is that external APIs are an externality, meaning that you have no control over them and that change over time should be expected with them.
- Use the latest Documentation from the API.

Overview | Maps JavaScript API | Google Developers

Google Maps (ii)

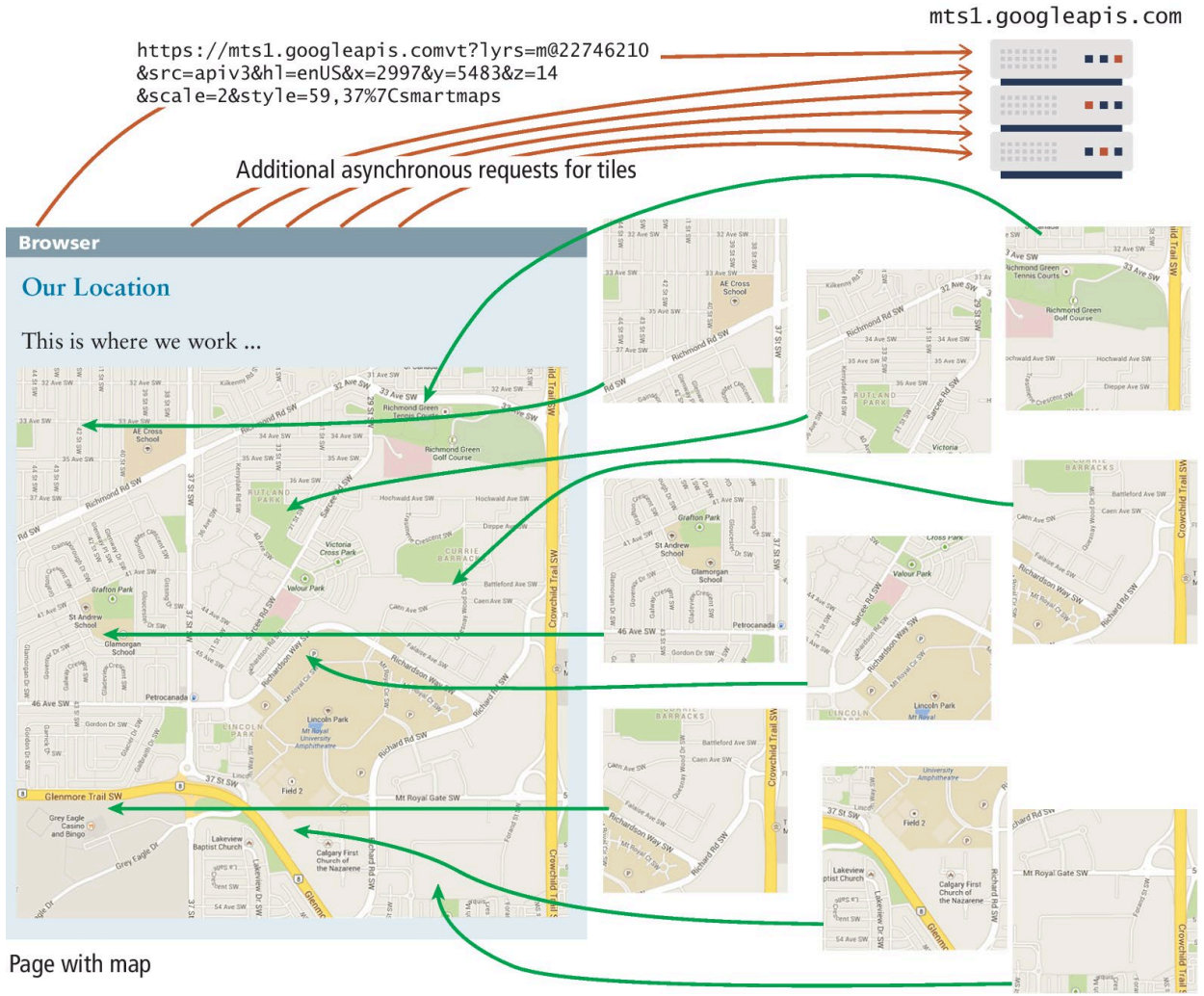
Notice that the API is made available to your page by referencing it in a **<script>** tag.

Our **initMap**() function creates a map object via the **google.maps.Map()** function constructor, which is defined within the library downloaded in our <script> element.

```
<head>
…
<style>
    #map {
        height: 500px;
    }
</style>
<script>
function initMap() {
    const map = new
    google.maps.Map(document.querySel
ector('#map'), {
    center: {lat: 51.011179,
                lng: -114.132866},
                zoom: 14
    });
}
</script>
<script
src="https://maps.googleapis.com/maps/
api/js?key=YOUR-API-
KEY&callback=initMap" async defer>
</script>
</head>
<body>
    Populating a Google Map
    <div id="map"></div>
</body>
</html>
```

LISTING 10.14 Webpage to output map centered on a location

Google Maps at work



Charting with Plotly.js

Charting is a common need for many websites. This section makes use of Plotly, which is open-source and available in a variety of other languages besides JavaScript.

Creating a simple chart is quite straightforward. Simply include the library, add an empty element that will contain the chart, and then make use of the newPlot() method, as shown in the following slide:

Charting with Plotly.js (ii)

```
<script>window.addEventListener("load", function() {
        const data = [
                                { x: [4,5,6,7,8,9,10,11],
                                 y: [23,25,13,15,10,13,17,20]
                                }
                        ];
        const layout = { title:'Simple Line Chart' };
        const options = { responsive: true };
        Plotly.newPlot("chartDiv", data, layout, options);
});
</script>
<script src="https://cdn.plot.ly/plotly-latest.min.js"></script>
<div id="chartDiv"></div>
```

Plotly.js display a simple line chart

```
window.addEventListener("load", async () => {
   const url =
'https://www.randyconnolly.com/funwebdev/3rd/api/
                stocks/sample-portfolio.json';
   try {
      let data = await fetch(url)
      .then(async response => await response.json() );
         generateChart( transformDataForCharting(data) );
      }
   catch (err) { console.error(err) }

   function transformDataForCharting(data) {…}
   function generateChart(portfolioData) {…}
});
```

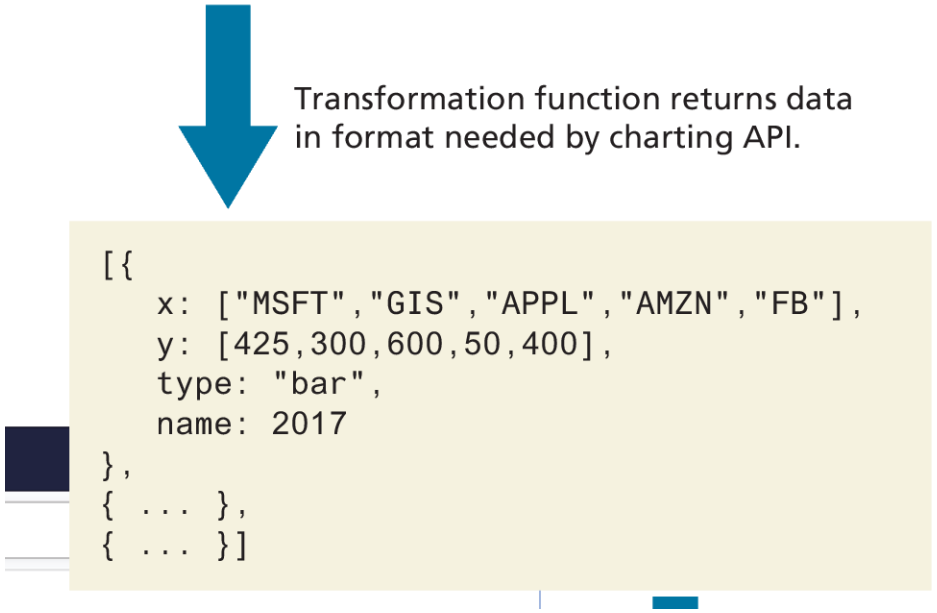
LISTING 10.15 Displaying a chart

JSON data received from external API.

```
[{
   "year": 2017,
   "portfolio": [{
      "symbol": "MSFT",
      "owned": 425
   }, {
      "symbol": "GIS",
      "owned": 300
   }, {
      "symbol": "APPL",
      "owned": 600
   }, {
      "symbol": "AMZN",
      "owned": 50
   }, {
      "symbol": "FB",
      "owned": 400
   }]
},
{
   "year": 2018,
   "portfolio": [ ... ]
},
{
   "year": 2019,
   "portfolio": [ ... ]
}]
```

Plotly.js display a simple line chart (ii)

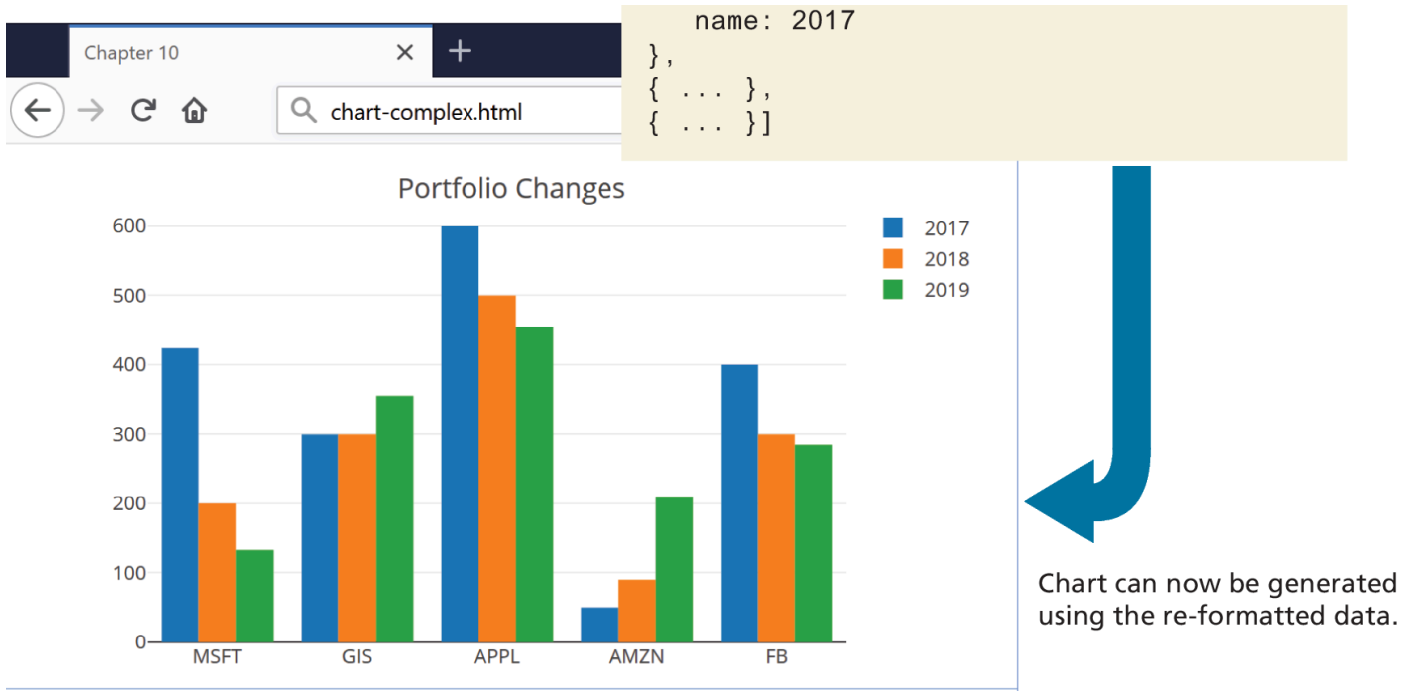
```
function transformDataForCharting(data) {
        const portfolioData = [];
        data.forEach((s) => {
                let trace = {};
                trace.x = [];
                trace.y = [];
                trace.type = 'bar’;
                trace.name = s.year;
                for (let p of s.portfolio) {
                        trace.x.push(p.symbol);
                        trace.y.push(p.owned);
                }
        portfolioData.push(trace);
        });
        return portfolioData;
}
```



LISTING 10.15 Displaying a chart

Plotly.js display a simple line chart (iii)

```
/* generate the chart */
function generateChart(portfolioData) {
        const layout = {
        title: 'Portfolio Changes',
        barmode: 'group'
        };
        const options = {
        responsive: true
        };
        Plotly.newPlot("chartDiv",
        portfolioData, layout, options);
}
```



LISTING 10.15 Displaying a chart

Key Terms

async . . . await

asynchronous code

browser API

card

class

cross-origin resource

sharing (CORS)

external API

fetch()

forEach()

find()

filter()

Geolocation API

localStorage

map()

module

origin

offline first

Progressive Web Applications (PWA)

prototype

promise

service workers

threads

TypeScript

web API

Web Storage API

Copyright